Summarize

Chapter 6. Pulling Apart Operational Data

Thursday, October 7, 08:55



Now that the Sysops Squad application was successfully broken into separately deployed domain services, Addison and Austen both realized that it was time to start thinking about breaking apart the monolithic Sysops Squad database. Addison agreed to start this effort, while Austen began to work on enhancing the CI/CD deployment pipeline. Addison met with Dana, the Sysops Squad data architect, and also Devon, one of the DBAs supporting the Penultimate Electronics databases.

"I'd like your opinions on how we might go about breaking up the Sysops Squad database," said Addison.

"Wait a minute," said Dana. "Who said anything about breaking apart the database?"

"Addison and I agreed last week that we needed to break up the Sysops Squad database," said Devon. "As you know, the Sysops Squad application has been going through a major overhaul, and breaking apart the data is part of that overhaul."

"I think the monolithic database is just fine," said Dana. "I see no reason why it should be broken apart. Unless you can convince me otherwise, I'm not going to budge on this issue. Besides, do you know how hard it would be to break apart that database?"

"Of course it will be difficult," said Devon, "but I know of a five-step process leveraging what are known as data domains that would work really well on this database. That way, we can even start investigating using different kinds of databases for certain parts of the application, like the knowledge base and even the customer survey functionality."

"Let's not get ahead of ourselves," said Dana. "And let's also not forget that I am the one who is responsible for all of these databases."

Addison quickly realized things were spiraling out of control, and quickly put some key negotiation and facilitation skills to use. "OK," said Addison, "we should have included you in

our initial discussions, and for that I apologize. I should have known better. What can we do to bring you on board and help us decompose the Sysops Squad database?"

"That's easy," said Dana. "Convince me that the Sysops Squad database really does need to be broken apart. Provide me with a solid justification. If you can do that, then we'll talk about Devon's five-step process. Otherwise, it stays as it is."

Breaking apart a database is hard—much harder, in fact, than breaking apart application functionality. Because data is generally the most important asset in the company, there is greater risk of business and application disruption when breaking apart or restructuring data. Also, data tends to be highly coupled to application functionality, making it harder to identify well-defined seams within a large data model.

In the same way a monolithic application is broken into separate deployment units, there are times when it is desirable (or even necessary) to break up a monolithic database as well. Some architecture styles, such as microservices, *require* data to be broken apart to form well-defined bounded contexts (where each service owns its own data), whereas other distributed architectures, such as service-based architecture, allow services to share a single database.

Interestingly enough, some of the same techniques used to break apart application functionality can be applied to breaking apart data as well. For example, components translate to data domains, class files translate to database tables, and coupling points between classes translate to database artifacts such as foreign keys, views, triggers, or even stored procedures.

In this chapter, we explore some of the drivers for decomposing data and show techniques for how to effectively break apart monolithic data into separate data domains, schemas, and even separate databases in an iterative and controlled fashion. Knowing that the database world is not all relational, we also discuss various types of databases (relational, graph, document, key-value, columnar, NewSQL, and cloud native) and outline the various trade-offs associated with each of these database types.

Data Decomposition Drivers

Breaking apart a monolithic database can be a daunting task, and as such it's important to understand if (and when) a database should be decomposed, as illustrated in Figure 6-1. Architects can justify a data decomposition effort by understanding and analyzing *data disintegrators* (drivers that justify breaking apart data) and *data integrators* (drivers that justify

keeping data together). Striving for a balance between these two driving forces and analyzing the trade-offs of each is the key to getting data granularity right.

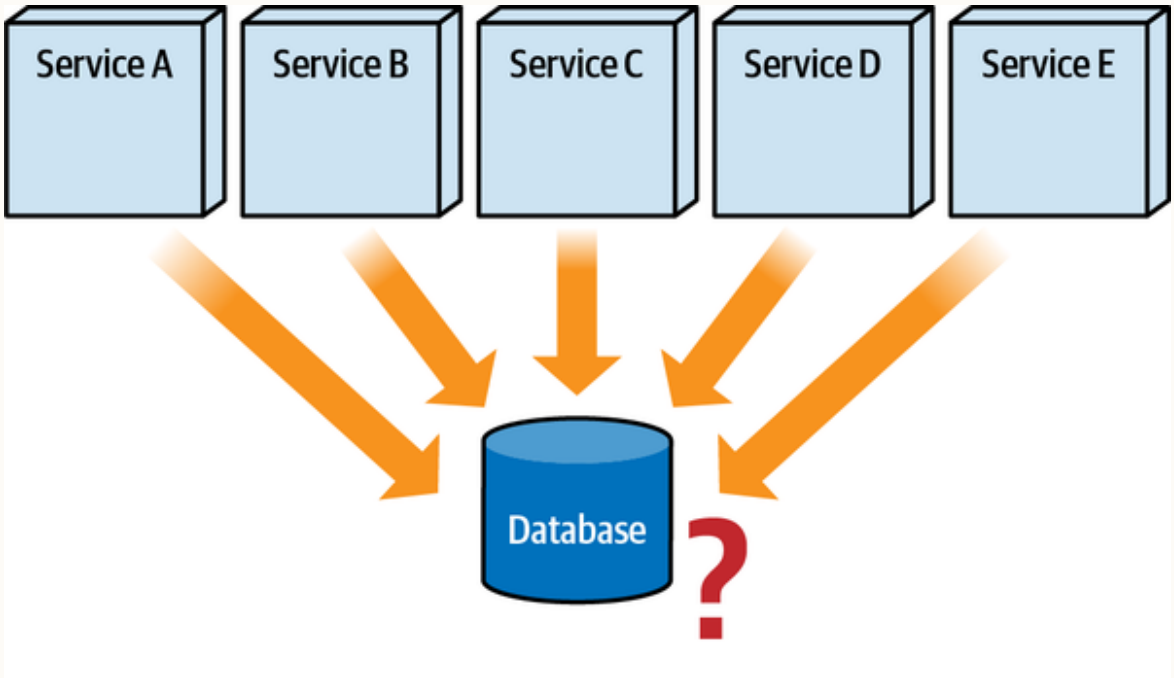


Figure 6-1. Under what circumstances should a monolithic database be decomposed?

In this section, we will explore the data disintegrators and data integrators used to help make the right choice when considering breaking apart monolithic data.

Data Disintegrators

Data disintegration drivers provide answers and justifications for the question "when should I consider breaking apart my data?" The six main disintegration drivers for breaking apart data include the following:

Change control

How many services are impacted by a database table change?

Connection management

Can my database handle the connections needed from multiple distributed services?

Scalability

Can the database scale to meet the demands of the services accessing it?

Fault tolerance

How many services are impacted by a database crash or maintenance downtime?

Architectural quanta

Is a single shared database forcing me into an undesirable single architecture quantum?

Database type optimization

Can I optimize my data by using multiple database types?

Each of these disintegration drivers is discussed in detail in the following sections.

Change control

One of the primary data disintegration drivers is controlling changes in the database table schemas. Dropping tables or columns, changing table or column names, and even changing the column type in a table break the corresponding SQL accessing those tables, and consequently break corresponding services using those tables. We call these types of changes *breaking changes* as opposed to adding tables or columns in a database, which generally do not impact existing queries or writes. Not surprisingly, change control is most impacted when using relational databases, but other database types can create change control issues as well (see "Selecting a Database Type").

As illustrated in Figure 6-2, when breaking changes occur to a database, multiple services must be updated, tested, and deployed together with the database changes. This coordination can quickly become both difficult and error prone as the number of separately deployed services sharing the same database increases. Imagine trying to coordinate 42 separately deployed services for a single breaking database change!

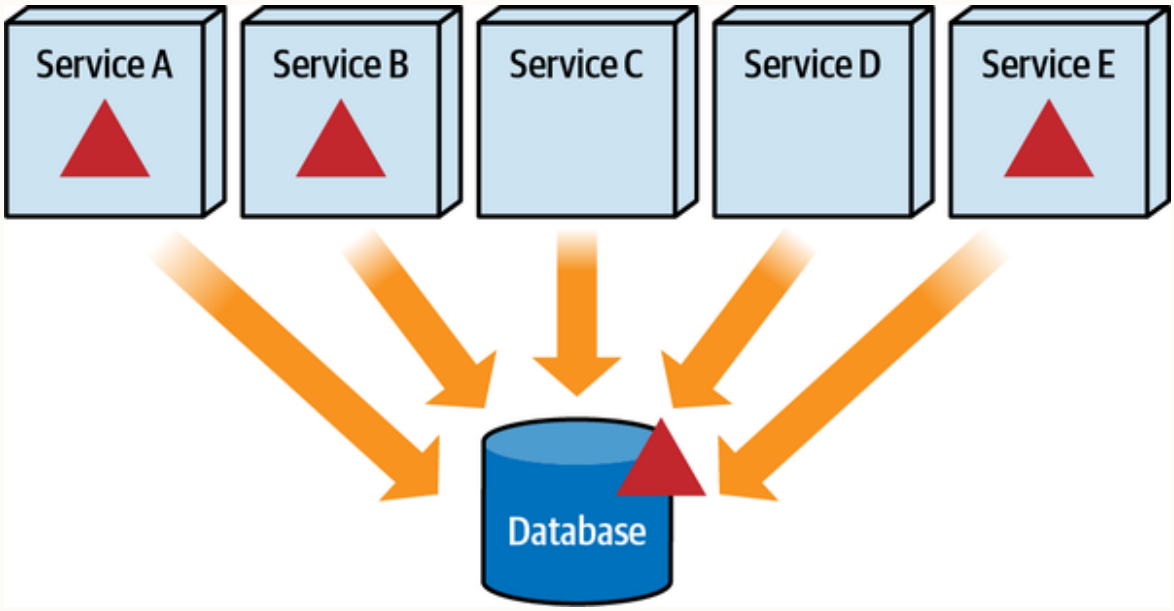


Figure 6-2. Services impacted by the database change must be deployed together with the database

Coordinating changes to multiple distributed services for a shared database change is only half the story. The real danger of changing a shared database in any distributed architecture is forgetting about services that access the table just changed. As illustrated in Figure 6-3,

those services become nonoperational *in production* until they can be changed, tested, and redeployed.

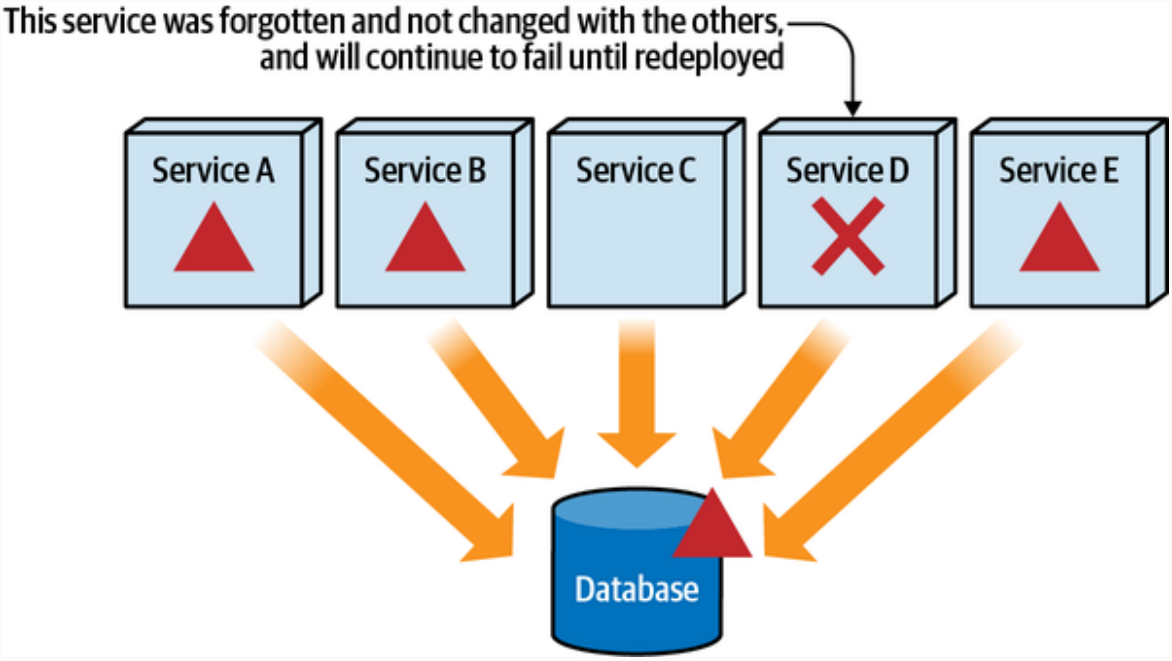


Figure 6-3. Services impacted by a database change but forgotten will continue to fail until redeployed

In most applications, the danger of forgotten services is mitigated by diligent impact analysis and agressive regression testing. However, consider a microservices ecosystem with 400 services, all sharing the same monolithic highly available clustered relational database. Imagine running around to all the development teams in many domain areas, trying to find out which services use the table being changed. Also imagine having to then coordinate, test, and deploy all of these services *together* as a single unit, along with the database. Thinking about this scenario starts to become a mind-numbing exercise, usually leading to some degree of insanity.

Breaking apart a database into well-defined bounded contexts significantly helps control breaking database changes. The bounded context concept comes from the seminal book *Domain-Driven Design* by Eric Evans (Addison-Wesley) and describes the source code, business logic, data structures, and data all bound together—encapsulated—within a specific context. As illustrated in Figure 6-4, well-formed bounded contexts around services and their corresponding data helps control change, because change is isolated to just those services within that bounded context.

Most typically, bounded contexts are formed around services and the data the services owns. By "own" we mean a service that writes to the database (as opposed to having read-only access to the data). We discuss distributed data ownership in more detail in Chapter 9.

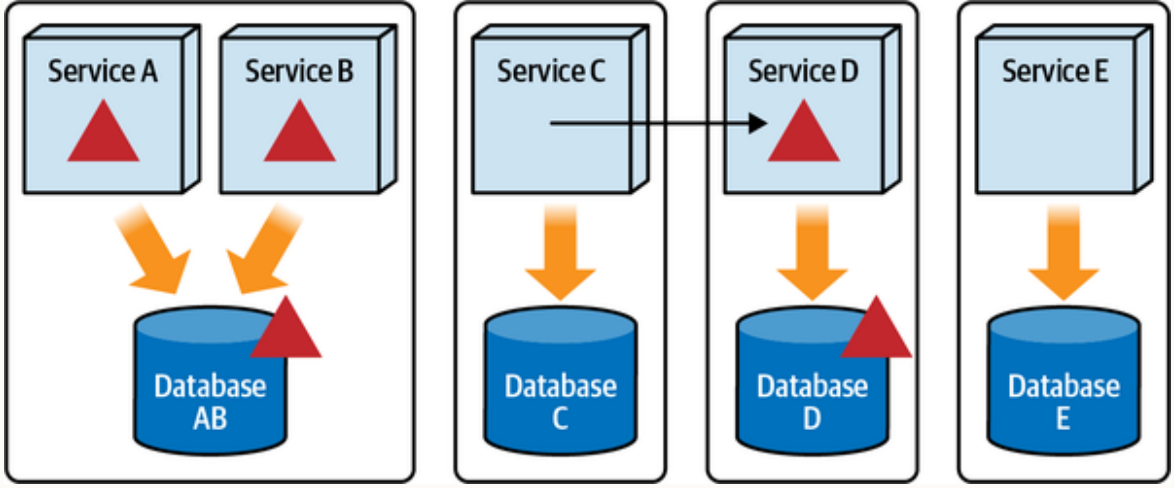


Figure 6-4. Database changes are isolated to only those services within the associated bounded context

Notice in Figure 6-4 that Service C needs access to some of the data in Database D that is contained in a bounded context with Service D. Since Database D is in a different bounded context, Service C cannot directly access the data. This would not only violate the bounded context rule, but also create a mess with regard to change control. Therefore, Service C must *ask* Service D for the data. There are many ways of accessing data a service doesn't own while still maintaining a bounded context. These techniques are discussed in detail in Chapter 10.

One important aspect of a bounded context related to the scenario between Service C needing data and Service D owning that data within its bounded context is that of *database abstraction*. Notice in Figure 6-5 that Service D is sending data that was requested by Service C through some sort of *contract* (such as JSON, XML, or maybe even an object).

The advantage of the bounded context is that the data sent to Service C can be a different contract than the schema for Database D. This means that a breaking change to some table in Database D impacts only Service D and not necessarily the contract of the data sent to Service C. In other words, Service C is abstracted from the actual schema structure of Database D.

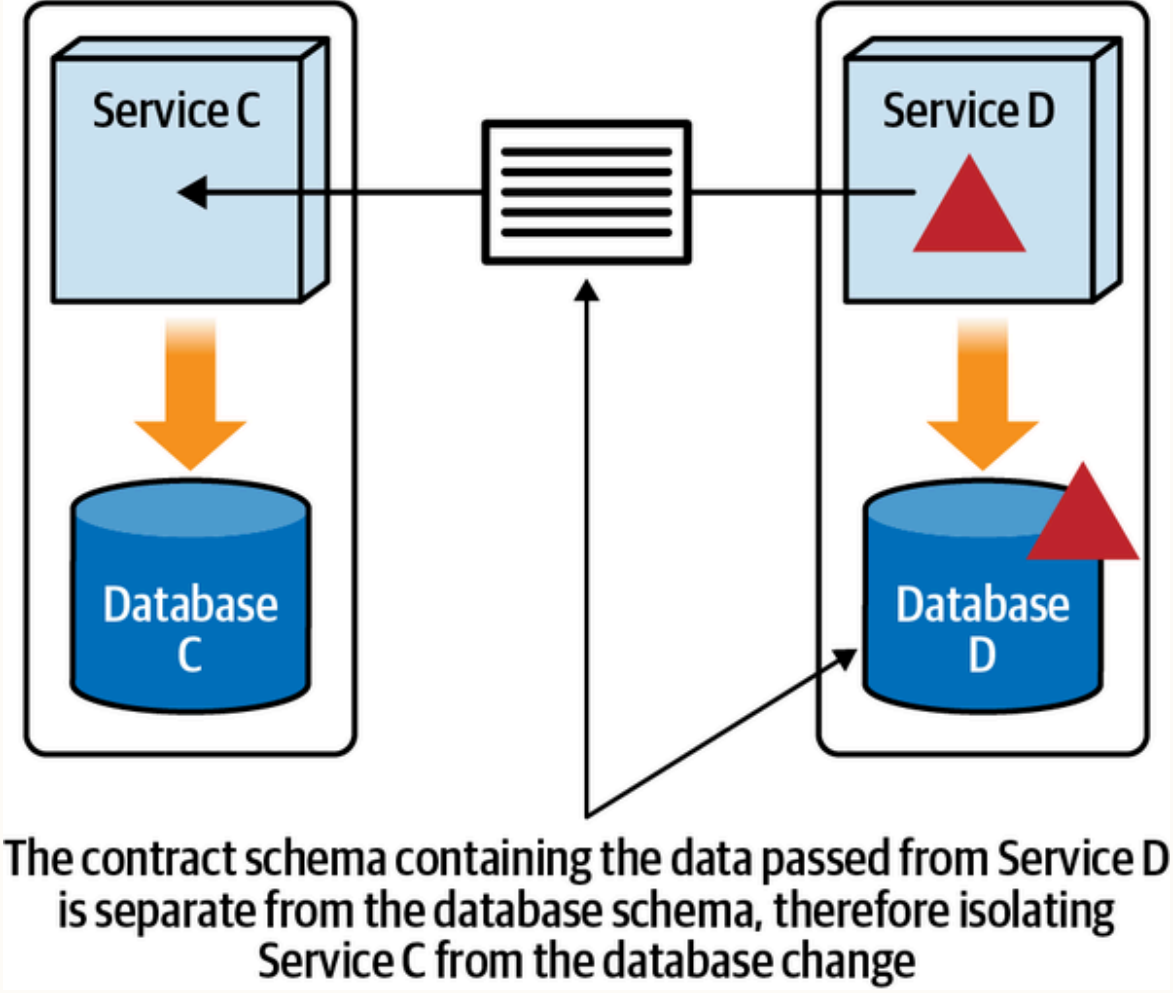


Figure 6-5. The contract from a service call abstracts the caller from the underlying database schema

To illustrate the power of this bounded context abstraction within a distributed architecture, assume Database D has a Wishlist table with the following structure:

```
CREATE TABLE Wishlist
(
CUSTOMER_ID VARCHAR(10),
ITEM_ID VARCHAR(20),
QUANTITY INT,
EXPIRATION_DT DATE
);
```

The corresponding JSON contract that Service D sends to Service C requesting wish list items is as follows:

```
{
  "$schema": "http://json-schema.org/draft-04/schema#",
  "properties": {
    "cust_id": {"type": "string"},
    "item_id": {"type": "string"},
    "qty": {"type": "number"},
    "exp_dt": {"type": "number"}
```

```
  },
}
```

Notice how the expiration data field (`exp_dt`) in the JSON schema is named differently than the database column name and is specified as a number (a long value representing the epoch time—the number of milliseconds since midnight on 1 January 1970), whereas in the database it is represented as a `DATE` field. Any column name change or column type change made in the database no longer impacts Service C because of the separate JSON contract.

To illustrate this point, suppose the business decides to no longer expire wish list items. This would require a change in the table structure of the database:

```
ALTER TABLE Wishlist
DROP COLUMN EXPIRATION_DT;
```

Service D would have to be modified to accommodate this change because it is within the same bounded context as the database, but the corresponding contract would not have to change at the same time. Until the contract is eventually changed, Service D could either specify a date far into the future or set the value to zero indicating the item doesn't expire. The bottom line is that Service C is abstracted from breaking changes made to Database D due to the bounded context.

Connection management

Establishing a connection to a database is an expensive operation. A database connection pool is often used not only to increase performance, but also to limit the number of concurrent connections an application is allowed to use. In monolithic applications, the database connection pool is usually owned by the application (or application server). However, in distributed architectures, each service—or more specifically, each service instance—typically has its own connection pool. As illustrated in Figure 6-6, when multiple services share the same database, the number of connections can quickly become saturated, particularly as the number of services or service instances increase.

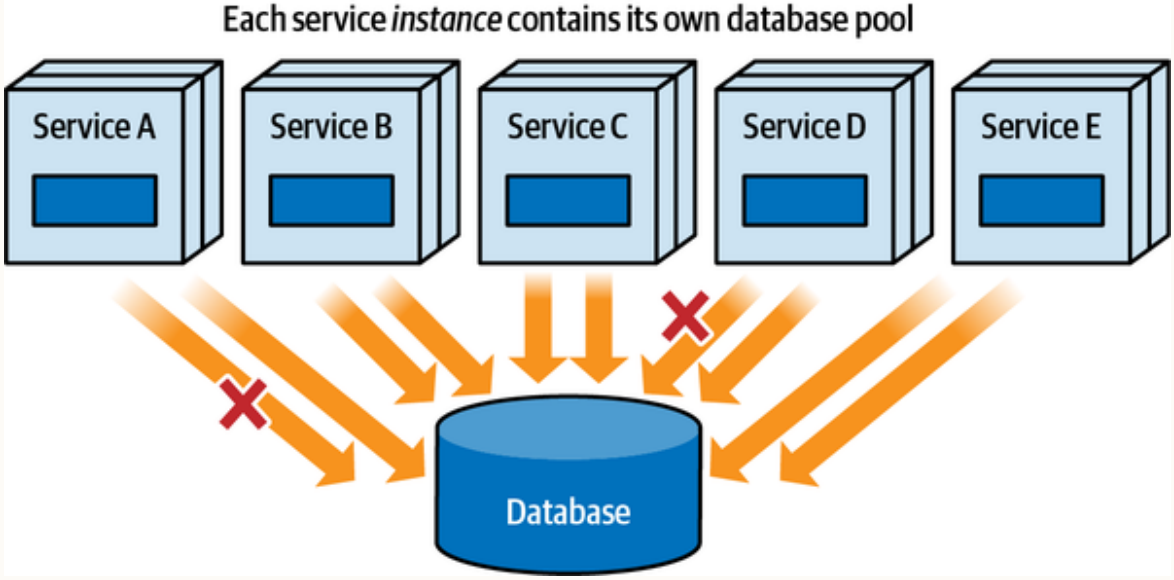


Figure 6-6. Database connections can quickly get saturated with multiple service instances

Reaching (or exceeding) the maximum number of available database connections is yet another driver to consider when deciding whether to break apart a database. Frequent connection waits (the amount of time it takes waiting for a connection to become available) is usually the first sign that the maximum number of database connections has been reached. Since connection waits can also manifest themselves as request time-outs or tripped circuit breakers, looking for connection waits is usually the first thing we recommend if these conditions frequently occur when using a shared database.

To illustrate the issues associated with database connections and distributed architecture, consider the following example: a monolithic application with 200 database connections is broken into a distributed architecture consisting of 50 services, each with 10 database connections in its connection pool.

Original monolithic application	200 connections
Distributed services	50
Connections per service	10
Minimum service instances	2
Total service connections	1,000

Notice how the number of database connections within the same application context grew from 200 to 1,000, and the services haven't even started scaling yet! Assuming half of the

services scale to an average of 5 instances each, the number of database connections quickly grows to 1,700.

Without some sort of connection strategy or governance plan, services will try to use as many connections as possible, frequently starving other services from much needed connections. For this reason, it's important to govern how database connections are used in a distributed architecture. One effective approach is to assign each service a *connection quota* to govern the distribution of available database connections across services. A connection quota specifies the maximum number of database connections a service is allowed to use or make available in its connection pool.

By specifying a connection quota, services are not allowed to create more database connections than are allocated to it. If a service reaches the maximum number of database connections in its quota, it must wait for one of the connections it's using to become available. This method can be implemented using two approaches: evenly distributing the same connection quota to every service, or assigning a different connection quota to each service based on its needs.

The even distribution approach is typically used when first deploying services, and it is not known yet how many connections each service will need during normal and peak operations. While simple, this approach is not overly efficient because some services may need more connections than others, while some connections held by other services may go unused.

While more complex, the variable distribution approach is much more efficient for managing database connections to a shared database. With this approach, each service is assigned a different connection quota based on its functionality and scalability requirements. The advantage of this approach is that it optimizes the use of available database connections across distributed services, making sure those services that require more database connections have them available for use. However, the disadvantage is that it requires knowledge about the nature of the functionality and the scalability requirements of each service.

We usually recommend starting out with the even distribution approach and creating fitness functions to measure the concurrent connection usage for each service. We also recommend keeping the connection quota values in an external configuration server (or service) so that the values can be easily adjusted either manually or programmatically through simple machine learning algorithms. This technique not only helps mitigate connection saturation risk, but also properly balances available database connections between distributed services to ensure that no idle connections are wasted.

Table 6-1 shows an example of starting out using the even distribution approach for a database that can support a maximum of 100 concurrent connections. Notice that Service A has

only ever needed a maximum of 5 connections, Service C only 15 connections, and Service E only 14 connections, whereas Service B and Service D have reached their max connection quota and have experienced connection waits.

Table 6-1. Connection quota allocations evenly distributed

	Service	Quota	Max used	Waits
	A	20	5	No
→	B	20	20	Yes
	C	20	15	No
→	D	20	20	Yes
	E	20	14	No

Since Service A is well below its connection quota, this is a good place to start reallocating connections to other services. Moving five database connections to Service B and five database connections to Service D yields the results shown in Table 6-2.

Table 6-2. Connection quota allocations with varying distributions

	Service	Quota	Max used	Waits
	A	10	5	No
→	B	25	25	Yes
	C	20	15	No
	D	25	25	No
	E	20	14	No

This is better, but Service B is still experiencing connection waits, indicating that it requires more connections than it has in its connection quota. Readjusting the quotas even further by taking two connections each from Service A and Service E yields much better results, as shown in Table 6-3.

Table 6-3. Further connection quota tuning results in no connection waits

Service	Quota	Max used	Waits
A	8	5	No
B	29	27	No
C	20	15	No
D	25	25	No
E	18	14	No

This analysis, which can be derived from continuous fitness functions that gather streamed metrics data from each service, can also be used to determine how close the maximum number of connections used is to the maximum number of connections available, and also how much buffer exists for each service in terms of its quota and maximum connections used.

Scalability

One of the many advantages of a distributed architecture is scalability—the ability for services to handle increases in request volume while maintaining a consistent response time. Most cloud-based and on-prem infrastructure-related products do a good job at ensuring that services, containers, HTTP servers, and virtual machines scale to satisfy increases in demand. But what about the database?

As illustrated in Figure 6-7, service scalability can put a tremendous strain on the database, not only in terms of database connections (as discussed in the prior section), but also on throughput and database capacity. In order for a distributed system to scale, *all* parts of the system need to scale—including the database.

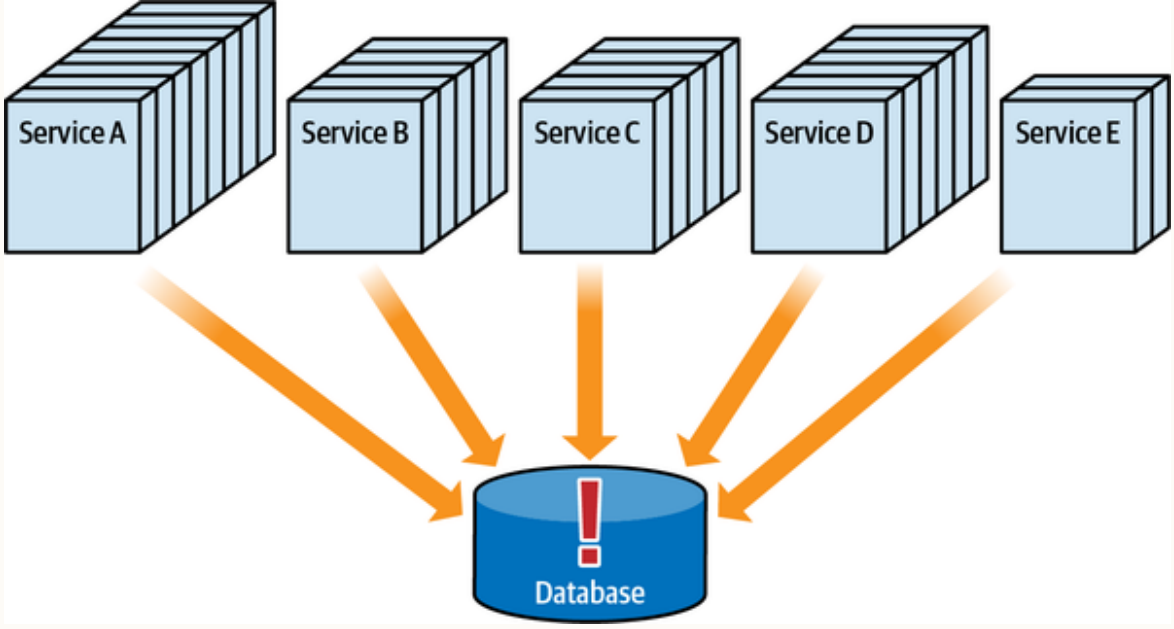


Figure 6-7. The database must also scale when services scale

Scalability is another data disintegration driver to consider when thinking about breaking apart a database. Database connections, capacity, throughput, and performance are all factors in determining whether a shared database can meet the demands of multiple services within a distributed architecture.

Consider the refined variable database connection quotas in Table 6-3 in the prior section. When services scale by adding multiple instances, the picture changes dramatically, as shown in Table 6-4, where the total number of database connections is 100.

Table 6-4. When services scale, more connection are used than are available

Service	Quota	Max used	Instances	Total used
A	8	5	2	10
B	29	27	3	81
C	20	15	3	45
D	25	25	2	50
E	18	14	4	56
TOTAL	100	86	14	242

Notice that even though the connection quota is distributed to match the 100 database connections available, once services start to scale, the quota is no longer valid because the

total number of connections used increases to 242, which is 142 more connections than are available in the database. This will likely result in connection waits, which in turn will result in overall performance degradation and request time-outs.

Breaking data into separate data domains or even a database-per-service, as illustrated in Figure 6-8, requires fewer connections to each database, hence providing better database scalability and performance as the services scale.

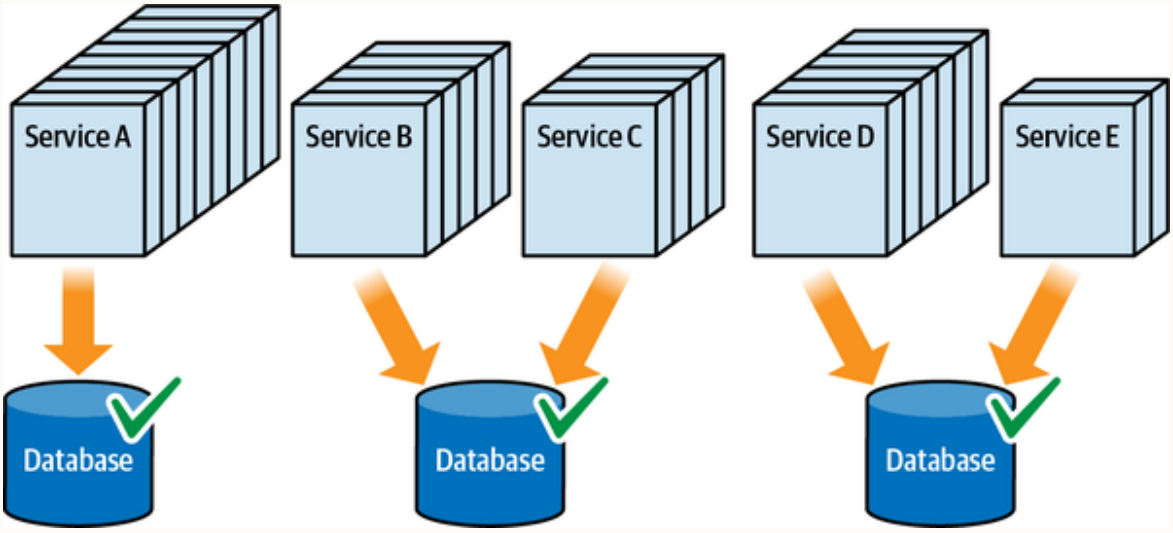


Figure 6-8. Breaking apart the database provides better database scalability

In addition to database connections, another factor to consider with respect to scalability is the load placed on the database. By breaking apart a database, less load is placed on each database, thereby also improving overall performance and scalability.

Fault tolerance

When multiple services share the same database, the overall system becomes less fault tolerant because the database becomes a single point of failure (SPOF). Here, we are defining fault tolerance as the ability of some parts of the system to continue uninterrupted when a service or database fails. Notice in Figure 6-9 that when sharing a single database, overall fault tolerance is low because if the database goes down, *all* services become nonoperational.

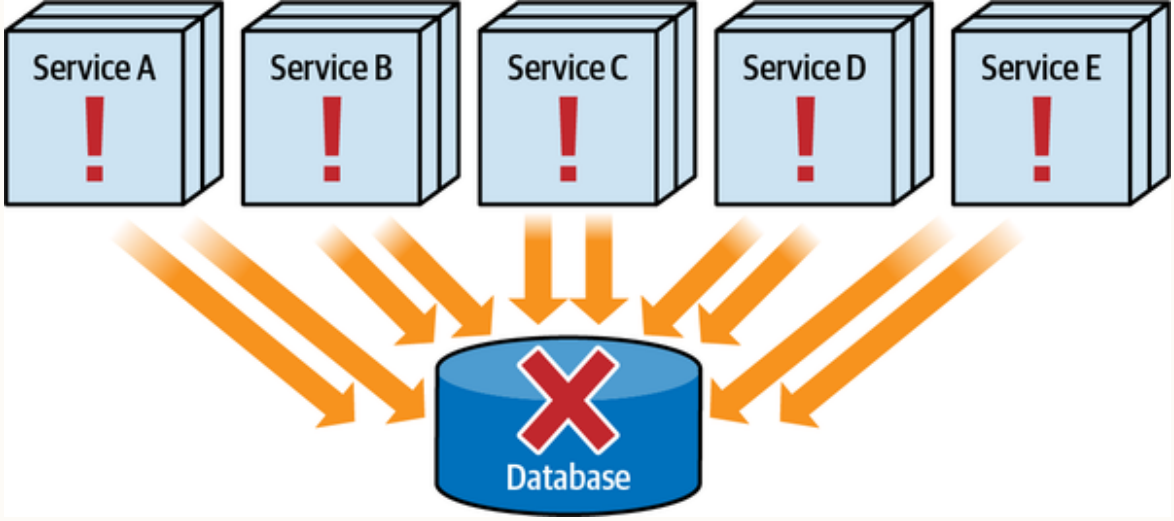


Figure 6-9. If the database goes down, all services become nonoperational

Fault tolerance is another driver for considering breaking apart data. If fault tolerance is required for certain parts of the system, breaking apart the data can remove the single point of failure in the system, as shown in Figure 6-10. This ensures that some parts of the system are still operational in the event of a database crash.

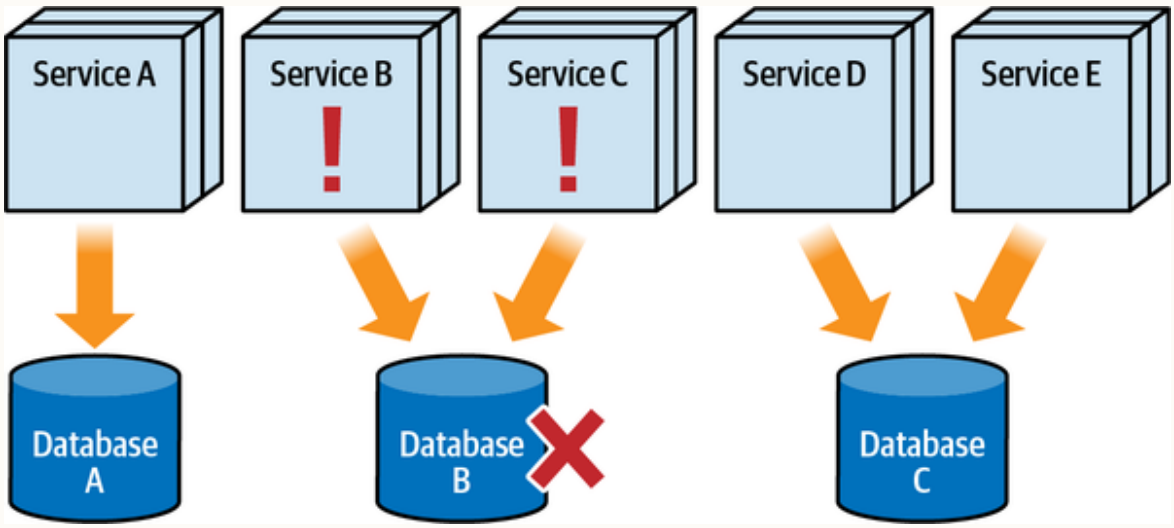


Figure 6-10. Breaking apart the database achieves better fault tolerance

Notice that since the data is now broken apart, if Database B goes down, only Service B and Service C are impacted and become nonoperational, whereas the other services continue to operate uninterrupted.

Architectural quantum

Recall from Chapter 2 that an architectural quantum is defined as an independently deployable artifact with high functional cohesion, high static coupling, and synchronous dynamic coupling. The architecture quantum helps provide guidance in terms of when to break apart a database, making it another data disintegration driver.

Consider the services in Figure 6-11, where Service A and Service B require different architectural characteristics than the other services. Notice in the diagram that although Service

A and Service B are grouped together, they do not form a separate quantum from the other services because of a single shared database. Thus, all five services, along with the database, form a single architectural quantum.

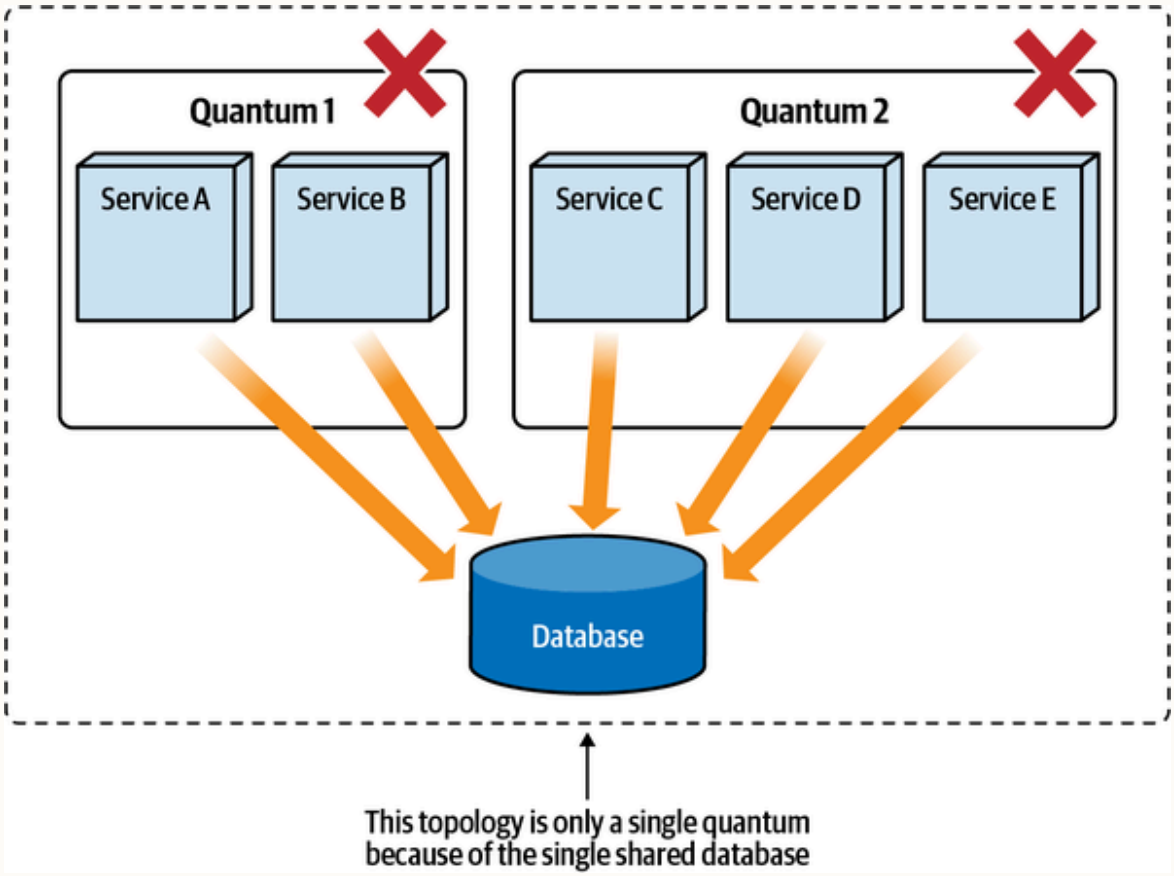


Figure 6-11. The database is part of the architectural quantum

Because the database is included in the *functional cohesion* part of the architecture quantum definition, it is necessary to break apart the data so that each resulting part can be in its own quantum. Notice in Figure 6-12 that since the database is broken apart, Service A and Service B, along with the corresponding data, are now a separate quantum from the one formed with services C, D, and E.

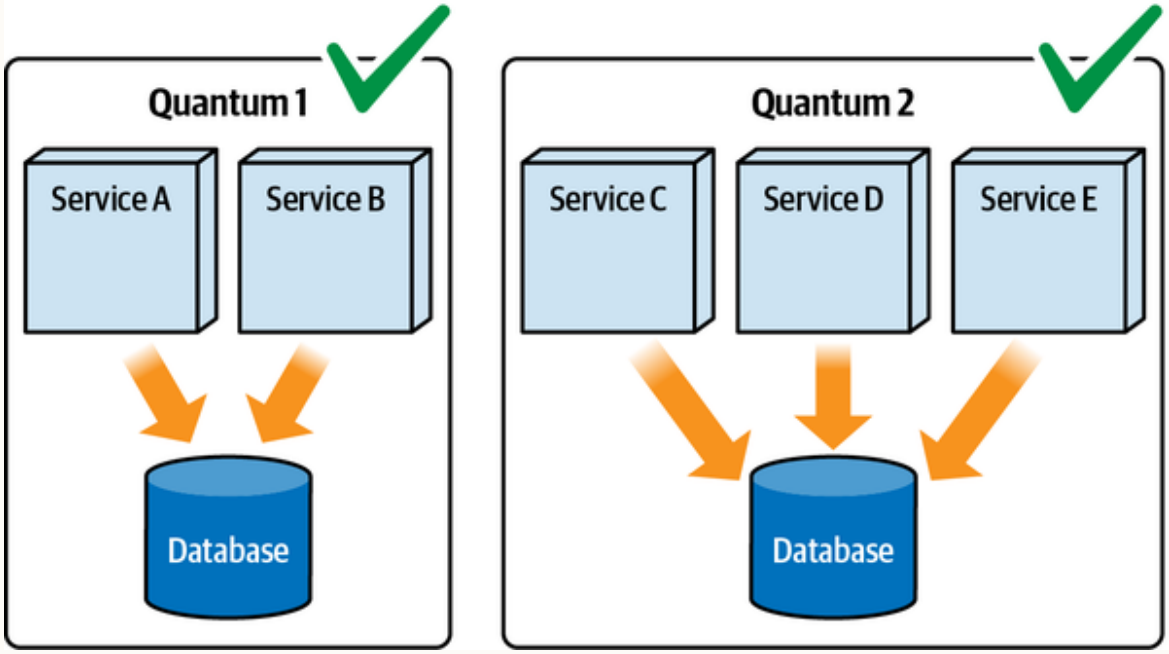


Figure 6-12. Breaking up the database forms two architectural quanta

Database type optimization

It's often the case that not all data is treated the same. When using a monolithic database, *all* data must adhere to that database type, therefore producing potentially sub-optimal solutions for certain types of data.

Breaking apart monolithic data allows the architect to move certain data to a more optimal database type. For example, suppose a monolithic relational database stored application-related transactional data, including reference data in the form of key-value pairs (such as country codes, product codes, warehouse codes, and so on). This type of data is difficult to manage in a relational database because the data is not relational in nature, but rather key-value. Hence, a *key-value* database (see "Key-Value Databases") would produce a more optimal solution than a relational database.

Data Integrators

Data integrators do the exact opposite of the data disintegrators discussed in the prior section. These drivers provide answers and justifications for the question "when should I consider putting data back together?" Along with data disintegrators, data integrators provide the balance and trade-offs for analyzing when to break apart data and when not to.

The two main integration drivers for pulling data back together are the following:

Data relationships

Are there foreign keys, triggers, or views that form close relationships between the tables?

Database transactions

Is a single transactional unit of work necessary to ensure data integrity and consistency?

Each of these integration drivers is discussed in detail in the following sections.

Data relationships

Like components within an architecture, database tables can be coupled as well, particularly with regard to relational databases. Artifacts like foreign keys, triggers, views, and stored procedures tie tables together, making it difficult to pull data apart; see Figure 6-13.

Imagine walking up to your DBA or data architect and telling them that since the database must be broken apart to support tightly formed bounded contexts within a microservices ecosystem, every foreign key and view in the database needs to be removed! That's not a likely (or even feasible) scenario, yet that is precisely what would need to happen to support a database-per-service pattern in microservices.

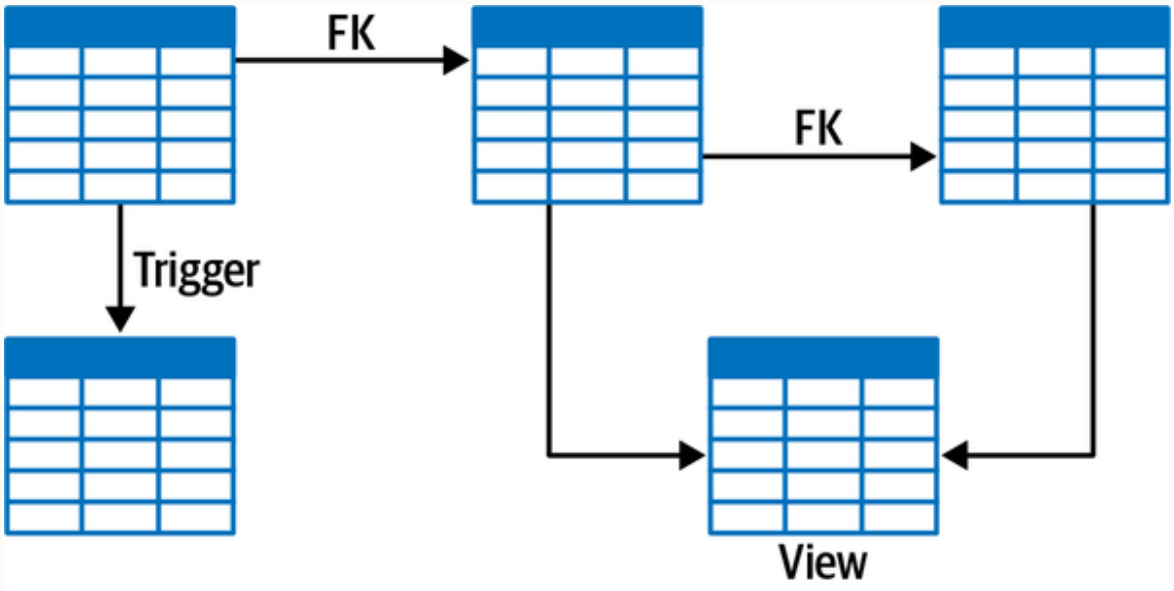


Figure 6-13. Foreign keys (FK), triggers, and views create tightly coupled relationships between data

These artifacts are necessary in most relational databases to support data consistency and data integrity. In addition to these physical artifacts, data may also be logically related, such as a problem ticket table and its corresponding problem ticket status table. However, as illustrated in Figure 6-14, these artifacts must be removed when moving data to another schema or database to form bounded contexts.

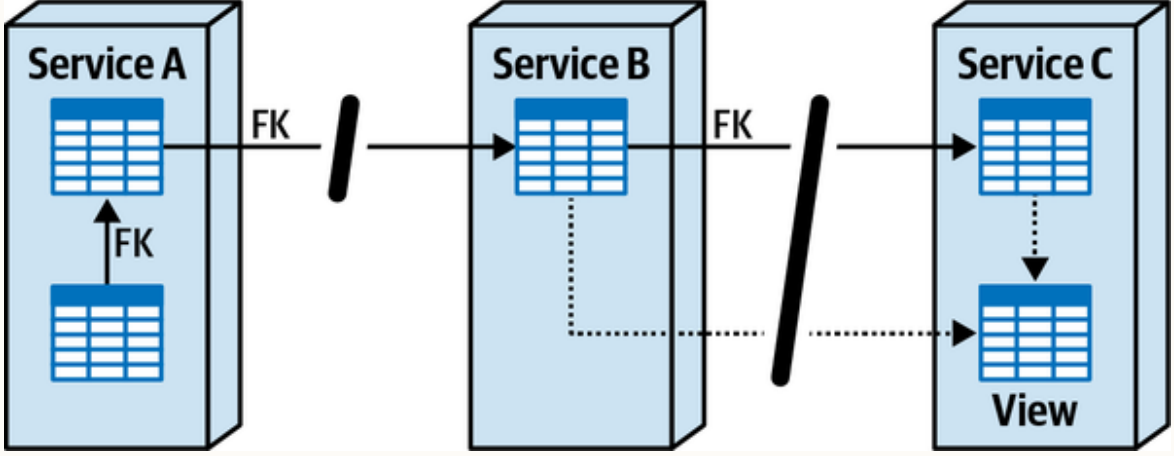


Figure 6-14. Data artifacts must be removed when breaking apart data

Notice that the foreign key (FK) relationship between the tables in Service A can be preserved because the data is in the same bounded context, schema, or database. However, the foreign keys (FK) between the tables in Service B and Service C must be removed (as well as the view that is used in Service C) because those tables are associated with different databases or schemas.

The relationship between data, either logical or physical, is a data integration driver, thus creating a trade-off between data disintegrators and data integrators. For example, is change control (a data disintegrator) more important than preserving the foreign key relationships between the tables (a data integrator)? Is fault tolerance (a data disintegrator) more important than preserving materialized views between tables (a data integrator)? Identifying what is more important helps make the decision about whether the data should be broken apart and what the resulting schema granularity should be.

Database transactions

Another data integrator is that of database transactions, something we discuss in detail in "Distributed Transactions". As shown in Figure 6-15, when a single service does multiple database write actions to separate tables in the same database or schema, those updates can be done within an Atomicity, Consistency, Isolation, Durability (ACID) transaction and either committed or rolled back as a single unit of work.

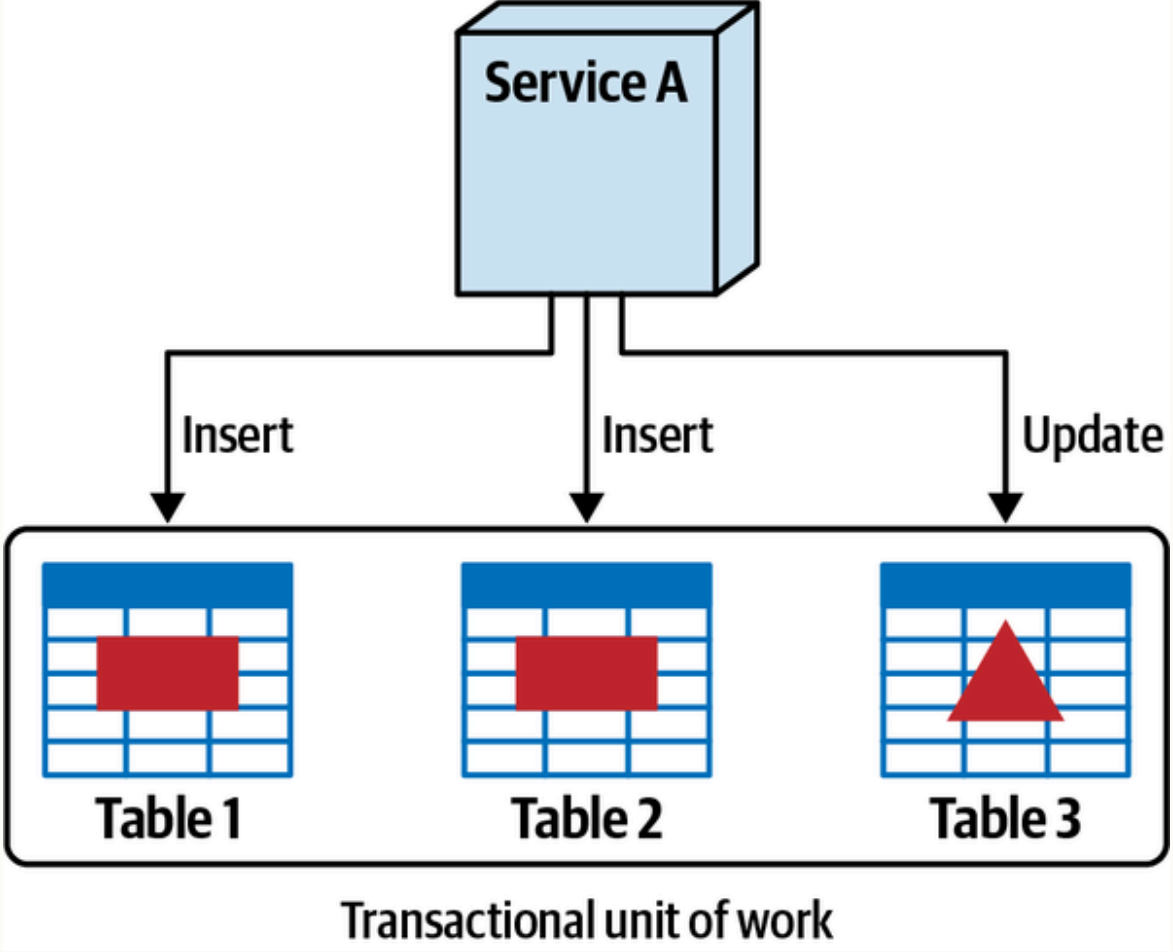


Figure 6-15. A single transactional unit of work exists when the data is together

However, when data is broken apart into either separate schemas or databases, as illustrated in Figure 6-16, a single transactional unit of work no longer exists because of the remote calls between services. This means that an insert or update can be committed in one table, but not in the other tables because of error conditions, resulting in data consistency and integrity issues.

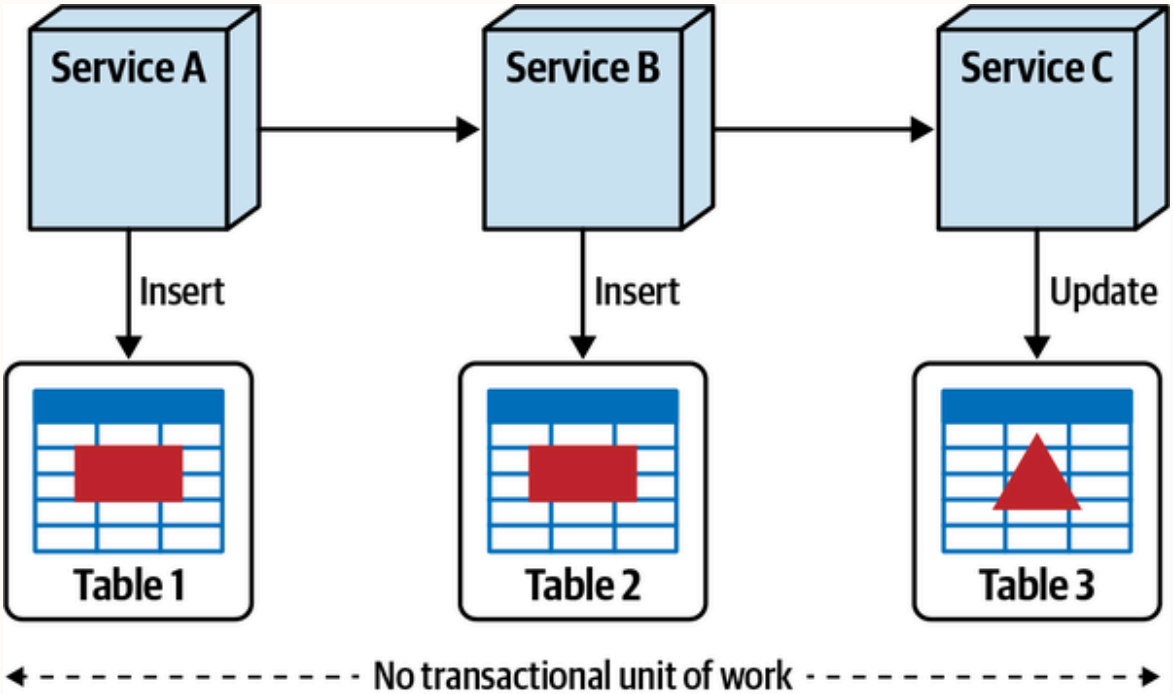


Figure 6-16. Single unit of work transactions don't exist when data is broken apart

While we dive into the details of distributed transaction management and transactional sagas in Chapter 12, the point here is to emphasize that database transactions are yet another data integration driver, and should be taken into account when considering breaking apart a database.

Sysops Squad Saga: Justifying Database Decomposition

Monday, November 15, 15:55



Armed with their justifications, Addison and Devon met to convince Dana that it was necessary to break apart the monolithic Sysops Squad database.

"Hi, Dana," said Addison. "We think we have enough evidence to convince you that it's necessary to break apart the Sysops Squad database."

"I'm all ears," said Dana, arms crossed and ready to argue that the database should remain as is.

"I'll start," said Addison. "Notice how these logs continuously show that whenever the operational reports run, the ticketing functionality in the application freezes up?"

"Yeah," said Dana, "I'll admit that even I suspected that. It's clearly something wrong with the way the ticketing functionality is accessing the database, not reporting."

"Actually," said Addison, "it's a combination of both ticketing *and* reporting. Look here."

Addison showed Dana metrics and logs that demonstrated some of the queries were necessarily wrapped in threads, and that the queries from the ticketing functionality were timing out because of a wait state when the reporting queries were run. Addison also showed how the reporting part of the system used parallel threads to query parts of the more complex reports concurrently, essentially taking up all of the database connections.

"OK, I can see how having a separate reporting database would help the situation from a database connection perspective. But that still doesn't convince me that the nonreporting data should be broken apart," said Dana.

"Speaking of database connections," said Devon, "look at this connection pool estimate as we start breaking apart the domain services."

Devon showed Dana the number of estimated services in the final planned Sysops Squad distributed application, including the projected number of instances for each of the services as the application scales. Dana explained to Devon that the connection pool was contained

within each separate service instance, not like in the current phase of the migration where the application server owned the connection pool.

"So you see, Dana," said Devon, "with these projected estimates, we will need an additional 2,000 connections to the database to provide the scalability we need to handle the ticket load, and we simply do not have them with a single database."

Dana took a moment to look over the numbers. "Do you agree with these numbers, Addison?"

"I do," said Addison. "Devon and I came up with them ourselves after a lot of analysis based on the amount of HTTP traffic as well as the projected growth rates supplied by Parker."

"I must admit," said Dana, "this is good stuff you've both prepared. I particularly like that you've already thought about not having services connect to multiple databases or schemas. As you know, in my book that's a no-go."

"Us, too. However, we have one more justification to talk to you about," said Addison. "As you may or may not know, we've been having lots of issues with regard to the system not being available for our customers. While breaking apart the services provides us with some level of fault tolerance, if a monolithic database should go down for either maintenance or a server crash, all services would become nonoperational."

"What Addison is saying," added Devon, "is that by breaking apart the database, we can provide better fault tolerance by creating domain silos for the data. In other words, if the survey database were to go down, ticketing functionality would still be available."

"We call that an architectural quantum," said Addison. "In other words, since the database is part of the static coupling of a system, breaking it apart would make the core ticketing functionality standalone and not synchronously dependent on other parts of the system."

"Listen," said Dana, "you've convinced me that there's good reasons to break apart the Sysops Squad database, but explain to me how you can even think about doing that. Do you realize how many foreign keys and views there are in that database? There's no way you're going to be able remove all of those things."

"We don't necessarily have to remove all of those artifacts. That's where data domains and the five-step process come into play," said Devon. "Here, let me explain..."

Decomposing Monolithic Data

Decomposing a monolithic database is hard, and requires an architect to collaborate closely with the database team to safely and effectively break apart the data. One particularly effective technique for breaking apart data is to leverage what is known as the *five-step process*. As illustrated in Figure 6-17, this evolutionary and iterative process leverages the concept of a data domain as a vehicle for methodically migrating data into separate schemas, and consequently different physical databases.

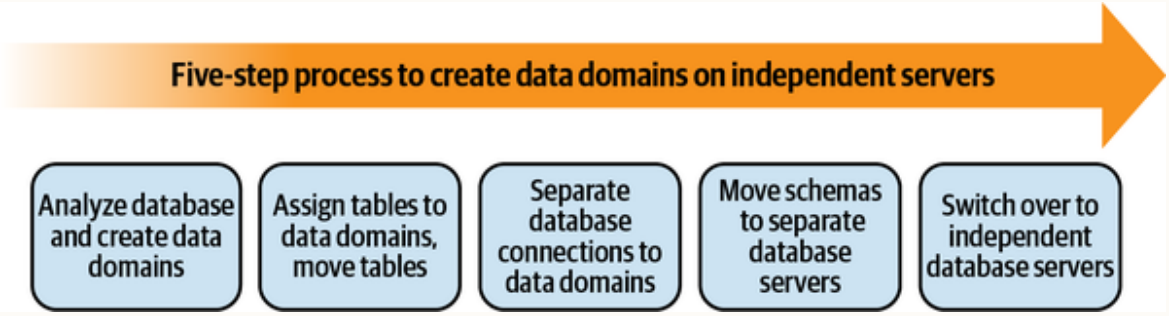


Figure 6-17. Five-step process for decomposing a monolithic database

A *data domain* is a collection of coupled database artifacts—tables, views, foreign keys, and triggers—that are all related to a particular domain and frequently used together within a limited functional scope. To illustrate the concept of a data domain, consider the Sysops Squad tables introduced in Table 1-2 and the corresponding proposed data domain assignments shown in Table 6-5.

Table 6-5. Existing Sysops Squad database tables assigned to data domains

Table	Proposed data domains
customer	Customer
customer_notification	Customer
survey	Survey
question	Survey
survey_administered	Survey
survey_question	Survey
survey_response	Survey
billing	Payment
contract	Payment
payment_method	Payment
payment	Payment
sysops_user	Profile
profile	Profile
expert_profile	Profile
expertise	Profile
location	Profile
article	Knowledge Base
tag	Knowledge Base

Table	Proposed data domains
keyword	Knowledge Base
article_tag	Knowledge Base
article_keyword	Knowledge Base
ticket	Ticketing
ticket_type	Ticketing
ticket_history	Ticketing

Table 6-5 lists six data domains within the Sysops Squad application: Customer, Survey, Payment, Profile, Knowledge base, and Ticketing. The `billing` table belongs to the Payment data domain, `ticket` and `ticket_type` tables belong to the Ticketing data domain, and so on.

One way to conceptually think about data domains is to think about the database as a soccer ball, where each white hexagon represents a separate data domain. As illustrated in Figure 6-18, each white hexagon of the soccer ball contains a collection of domain-related tables along with all of the coupling artifacts (such as foreign keys, views, stored procedures, and so on).

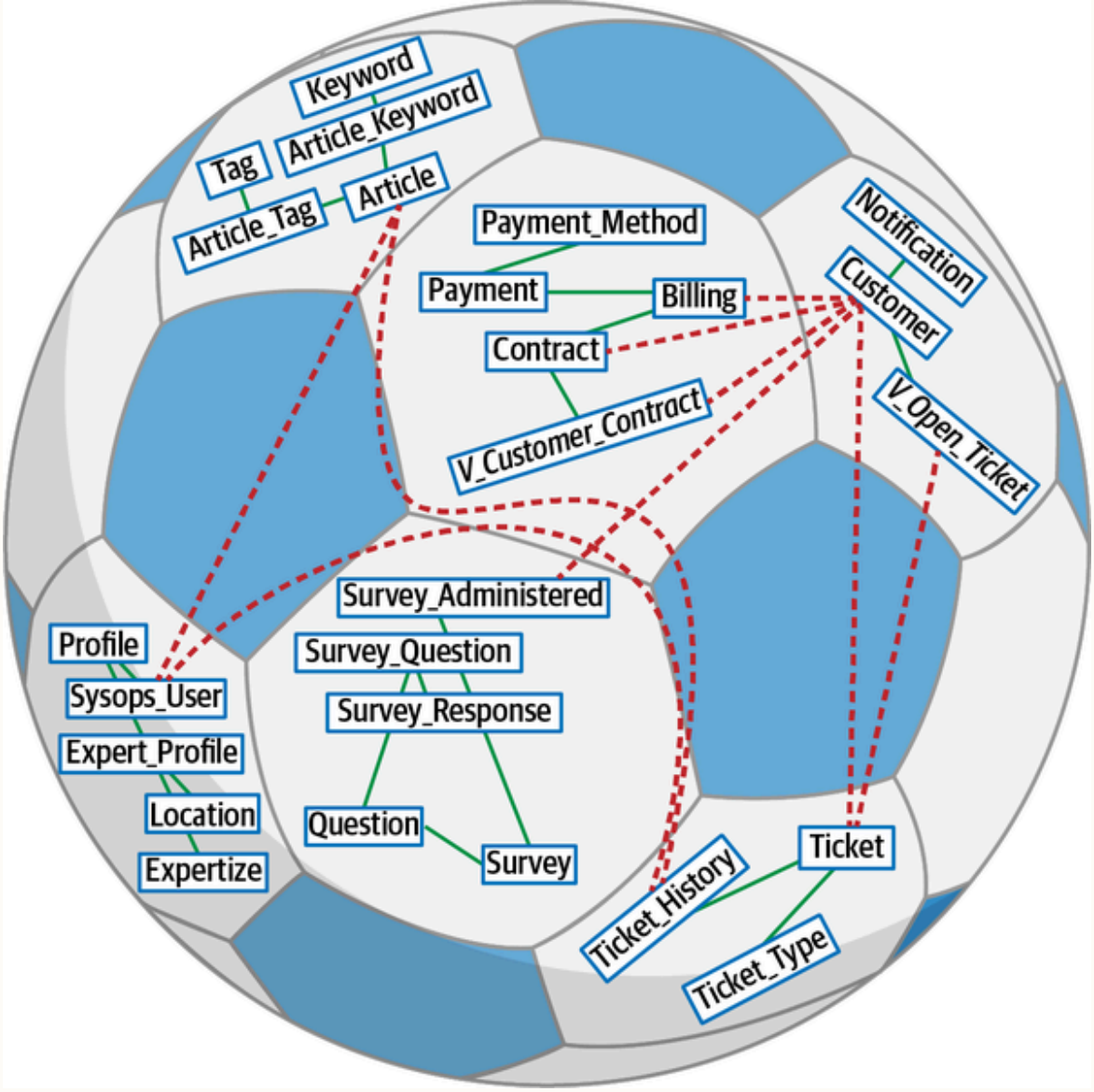


Figure 6-18. Database objects in a hexagon belong in a data domain

Visualizing the database this way allows the architect and database team to clearly see data domain boundaries and also the cross-domain dependencies (such as foreign keys, views, stored procedures, and so on) that need to be broken. Notice in Figure 6-18 that *within* each white hexagon, all data table dependencies and relationships can be preserved, but not *between* each white hexagon. For example, in the diagram notice that solid lines represent dependencies that are self-contained to the data domain, while the dotted lines cross data domains and must be removed when the data domains are extracted into separate schemas.

When extracting a data domain, these cross-domain dependencies must be removed. This means removing foreign-key constraints, views, triggers, functions, and stored procedures between data domains. Database teams can leverage the refactoring patterns found in the book *Refactoring Databases: Evolutionary Database Design*, by Scott Ambler and Pramod Sadalage (Addison-Wesley), to safely and iteratively remove these data dependencies.

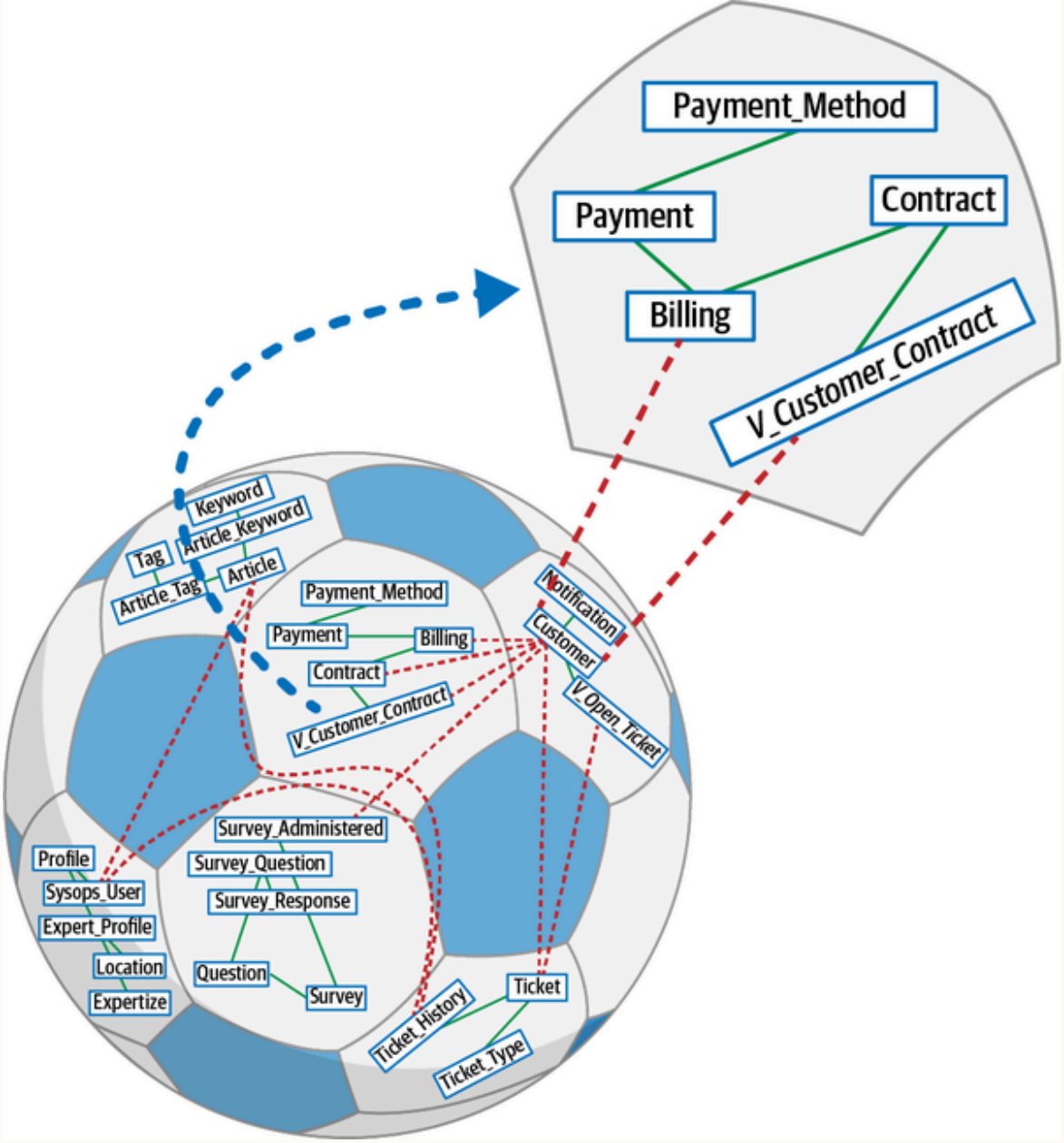


Figure 6-19. Tables belonging to data domains, extracted out, and connections that need to be broken

To illustrate the process of defining a data domain and removing cross-domain references, consider the diagram in Figure 6-19, where a data domain representing Payment is created. Since the `customer` table belongs to a different data domain than the `v_customer_contract`, the `customer` table must be removed from the view in the Payment domain. The original view `v_customer_contract` prior to defining the data domain is defined in Example 6-1.

Example 6-1. Database view to provide payment information to remove cross-domain references

```
CREATE VIEW [payment].[v_customer_contract]
 AS
SELECT
   customer.customer_id, customer.customer_name,
   contract.contract_start_date, contract.contract_duration,
   billing.billing_date, billing.billing_amount
FROM payment.contract AS contract
```

```
INNER JOIN customer.customer AS customer
   ON ( contract.customer_id = customer.customer_id )
INNER JOIN payment.billing AS billing
   ON ( contract.contract_id = billing.contract_id )
WHERE contract.auto_renewal = 0
```

Notice in the updated view shown in Example 6-2 that the join between `customer` and `payment` tables is removed, as is the column for the customer name (`customer.customer_name`).

Example 6-2. Database view to allow cross-domain access

```
CREATE VIEW [payment].[v_customer_contract]
 AS
SELECT
   billing.customer_id, contract.contract_start_date,
   contract.contract_duration, billing.billing_date,
   billing.billing_amount
FROM payment.contract AS contract
INNER JOIN payment.billing AS billing
   ON ( contract.contract_id = billing.contract_id )
WHERE contract.auto_renewal = 0
```

The bounded context rules for data domains apply just the same as individual tables—a service cannot talk to multiple data domains. Therefore, by removing this table from the view, the Payment service must now call the Customer service to get the customer name that it originally had from the view.

Once architects and database teams understand the concept of a data domain, they can apply the five-step process for decomposing a monolithic database. Those five steps are outlined in the following sections.

Step 1: Analyze Database and Create Data Domains

As illustrated in Figure 6-20, all services have access to all data in the database. This practice, known as the shared database integration style described by Gregor Hohpe and Bobby Woolf in their book *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions* (Addison-Wesley), creates a tight coupling between data and the services accessing that data. As discussed in "Data Decomposition Drivers", this tight coupling in the database makes change management very difficult.

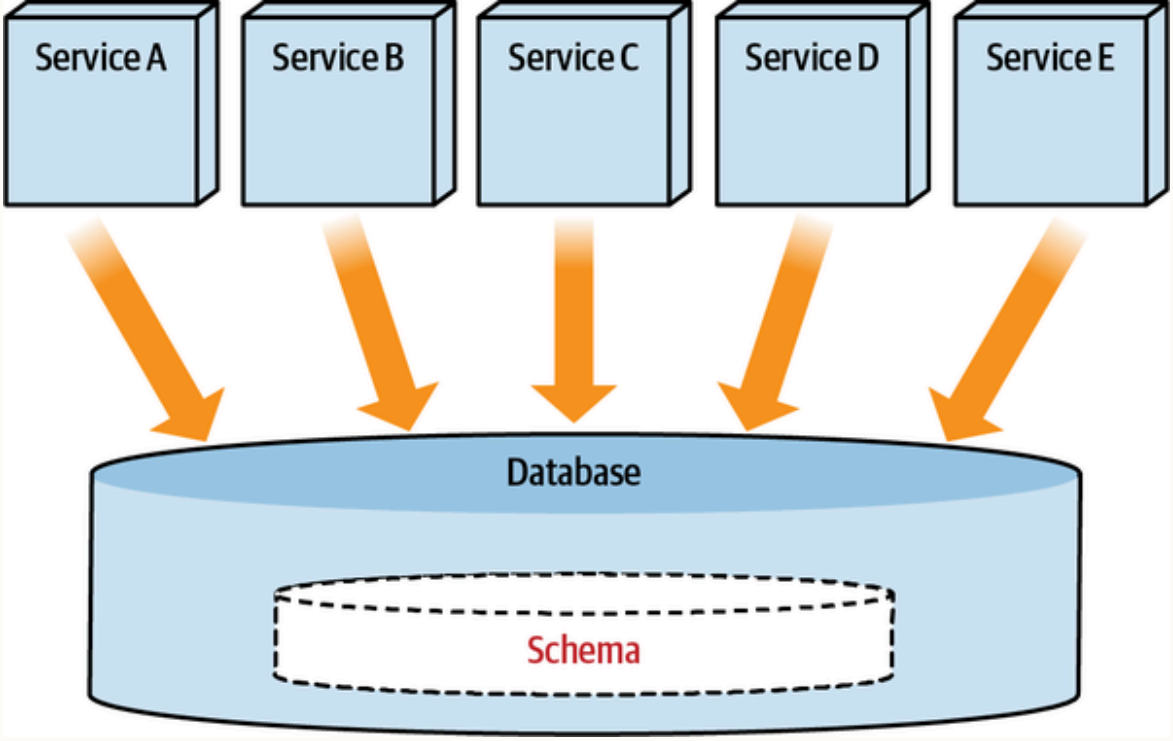


Figure 6-20. Multiple services use the same database, accessing all the tables necessary for read or write purposes

The first step in breaking apart a database is to identify specific domain groupings within the database. For example, as shown in Table 6-5, related tables are grouped together to help identify possible data domains.

Step 2: Assign Tables to Data Domains

The next step is to group tables along a specific bounded context, assigning tables that belong to a specific data domain into their own schema. A *schema* is a logical construct in database servers. A schema contain objects such as tables, views, functions, and so on. In some database servers, like Oracle, the schema is same as the user, while in other databases, like SQL Server, a schema is logical space for database objects where users have access to these schemas.

As illustrated in Figure 6-21, we have created schemas for each data domain and moved tables to the schemas to which they belong.

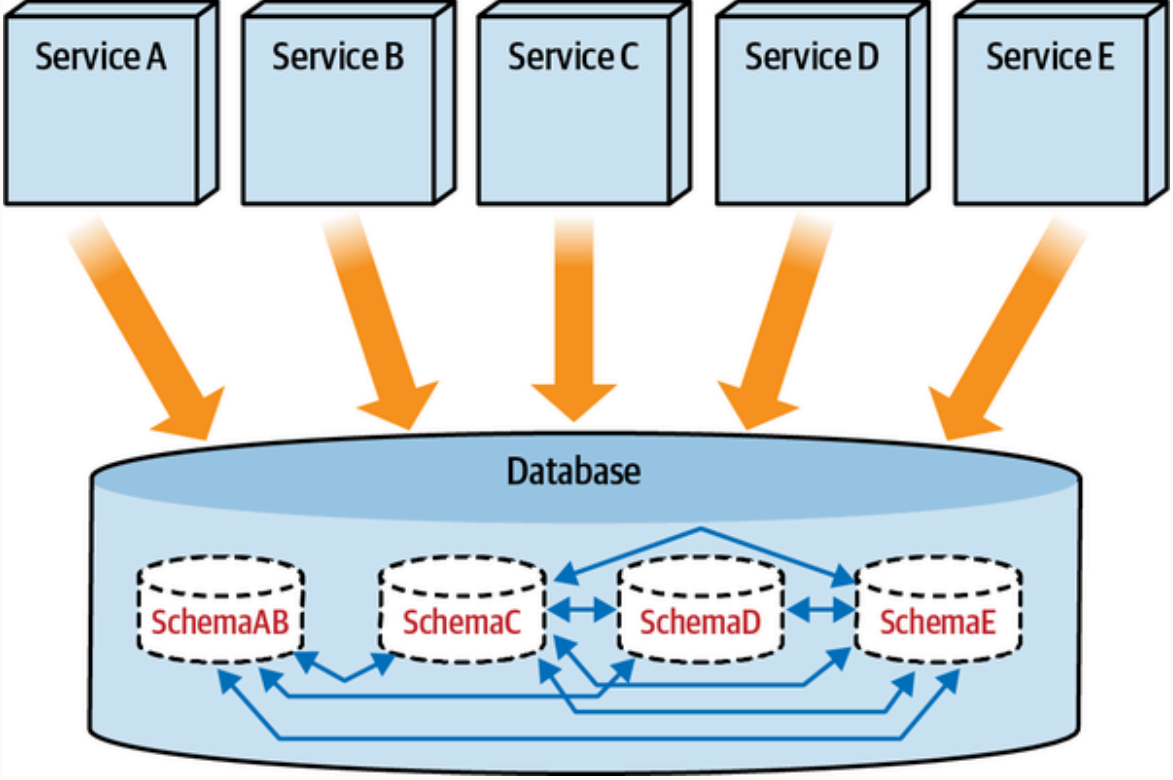


Figure 6-21. Services use the primary schema according to their data domain needs

When tables belonging to different data domains are tightly coupled and related to one another, data domains must necessarily be combined, creating a broader bounded context where multiple services own a specific data domain. Combining data domains is discussed in more detail in Chapter 9.

DATA DOMAIN VERSUS DATABASE SCHEMA

A *data domain* is an architectural concept, whereas a *schema* is a database construct that holds the database objects belonging to a particular data domain. While the relationship between a data domain and a schema is usually one to one, data domains can be mapped to one or more schemas, particularly when combining data domains because of tightly coupled data relationships. We will be referring to a data domain and schema to mean the same thing, and will be using the terms interchangeably.

To illustrate the assignment of tables to schemas, consider the Sysops Squad example where the `billing` table must be moved from its original schema to another data domain schema called `payment`:

```
ALTER SCHEMA payment TRANSFER sysops.billing;
```

Alternatively, a database team can create synonyms for tables that do not belong in their schema. *Synonyms* are database constructs, similar to *symlink*, that provide an alternate name for another database object that can exist in the same or different schema or server.

While the idea of synonyms is to eliminate cross-schema queries, read or write privileges are needed to access them.

To illustrate this practice, consider the following cross-domain query:

```
SELECT
   history.ticket_id, history.notes, agent.name
FROM ticket.ticket_history AS history
INNER JOIN profile.sysops_user AS agent
   ON ( history.assigned_to_sysops_user_id = agent.sysops_user_id )
```

Next, create a synonym for the `profile.sysops_user` table in the ticketing schema:

```
CREATE SYNONYM ticketing.sysops_user
FOR profile.sysops_user;
GO
```

As a result, the query can leverage the synonym `sysops_user` rather than the cross-domain table:

```
SELECT
   history.ticket_id, history.notes, agent.name
FROM ticket.ticket_history AS history
INNER JOIN ticket.sysops_user AS agent
   ON ( history.assigned_to_sysops_user_id = agent.sysops_user_id )
```

Unfortunately, creating synonyms this way for tables that are accessed across schemas provides the application developers with coupling points. To form proper data domains, these coupling points need to be broken apart at some later time, therefore moving the integration points from the database layer to the application layer.

While synonyms do not really get rid of cross-schema queries, they do allow for easier dependency checking and code analysis, making it easier to split these later on.

Step 3: Separate Database Connections to Data Domains

In this step, the database connection logic within each service is refactored to ensure services connect to a specific schema and have read and write access to the tables belonging only to their data domain. This transition, illustrated in Figure 6-22, is the most difficult since all cross-schema access must be resolved at the service level.

Notice that the database configuration has been changed so that all data access is done strictly via services and their connected schemas. In this example, Service C communicates with Service D and not with SchemaD. There is no cross-schema access; all synonyms created in "Step 2: Assign Tables to Data Domains" are removed.

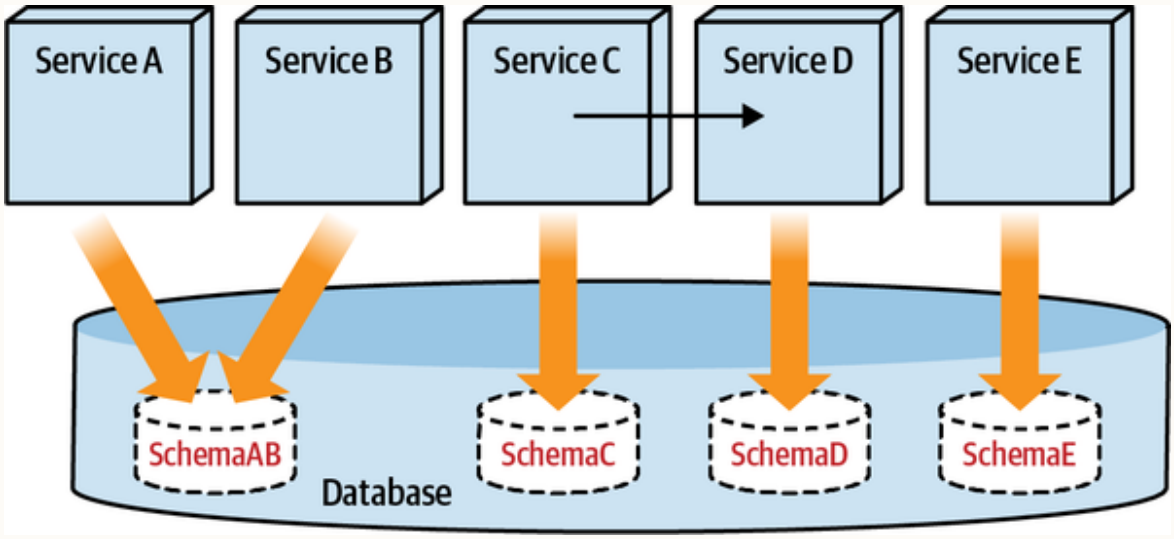


Figure 6-22. Move the cross-schema object access to the services, away from direct cross-schema access

---

TIP

When data from other domains is needed, do not reach into their databases. Instead, access it using the service that owns the data domain.

Upon completion of this step, the database is in a state of *data sovereignty per service*, which occurs when each service owns its own data. Data sovereignty per service is the nirvana state for a distributed architecture. Like all practices in architecture, it includes benefits and shortcomings:

Benefits

- Teams can change the database schema without worrying about affecting changes in other domains.
- Each service can use the database technology and database type best suitable for their use case.

Shortcomings

- Performance issues occur when services need access to large volumes of data.
- Referential integrity cannot be maintained in the database, resulting in the possibility of bad data quality.
- All database code (stored procedures, functions) that access tables belonging to other domains must be moved to the service layer.

Step 4: Move Schemas to Separate Database Servers

Once database teams have created and separated data domains, and have isolated services so that they access their own data, they can now move the data domains to separate physical databases. This is often a necessary step because even though services access their own schemas, accessing a single database creates a single *architecture quantum*, as discussed in Chapter 2, which might have adverse effects for operational characteristics, such as scalability, fault tolerance, and performance.

When moving schemas to separate physical databases, database teams have two options: backup and restore, or replication. These options are outlined as follows:

Backup and restore

With this option, teams first back up each schema with data domains, then set up database servers for each data domain. They then restore the schemas, connect services to schemas in the new database servers, and finally remove schemas from the original database server. This approach usually requires downtime for the migration.

Replicate

Using the replicate option, teams first set up database servers for each data domain. Next they replicate the schemas, switch connections over to the new database servers, and then remove the schemas from the original database server. While this approach avoids downtime, it does require more work to set up the replication and manage increased coordination.

Figure 6-23 shows an example of the replication option, where the database team sets up multiple database servers so that there is one database server for each data domain.

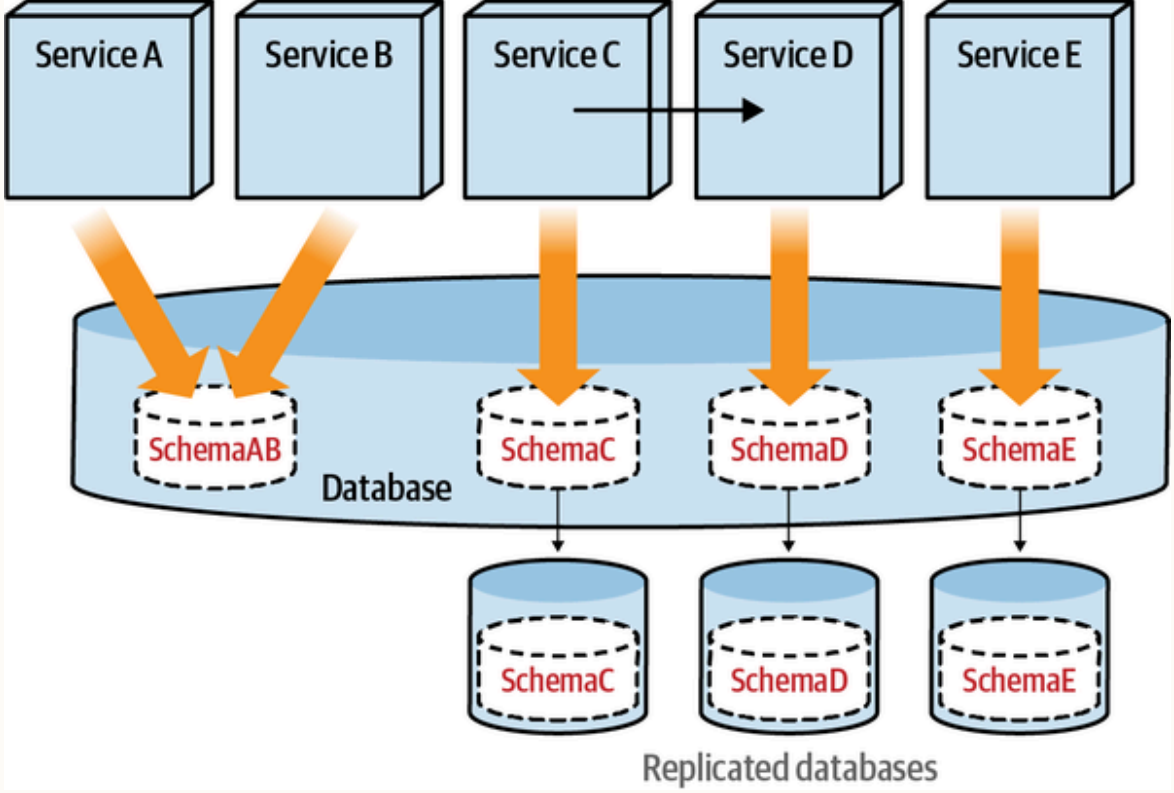


Figure 6-23. Replicate schemas (data domains) to their own database servers

Step 5: Switch Over to Independent Database Servers

Once the schemas are fully replicated, the service connections can be switched. The last step in getting the data domains and services to act as their own independent deployable units is to remove the connection to the old database servers and remove the schemas from the old database servers as well. The final state is seen in Figure 6-24.

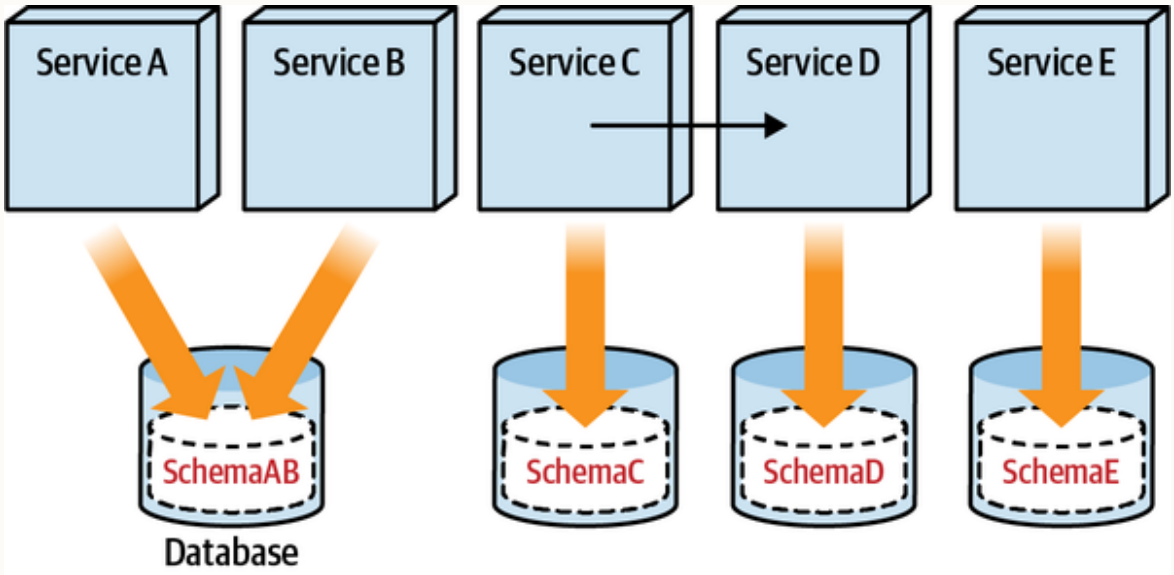


Figure 6-24. Independent database servers for each data domain

Once the database team has separated the data domains, isolated the database connections, and finally moved the data domains to their own database servers, they can optimize the individual database servers for availability and scalability. Teams can also analyze the

data to determine the most appropriate database type to use, introducing polyglot database usage within the ecosystem.

Selecting a Database Type

Beginning around 2005, a revolution has occurred in database technologies. Unfortunately, the number of products that have emerged during this time have created a problem known as The Paradox of Choice. Having such a large number of products and choices means having more trade-off decisions to make. Given that each product is optimized for certain trade-offs, it rests on both software and data architects to pick the appropriate product with these trade-offs in mind as it relates to their problem space.

In this section, we introduce star ratings for the various database types, using the following characteristics in our analysis:

Ease-of-learning curve

This characteristic refers to the ease with which new developers, data architects, data modelers, operational DBAs, and other users of the databases can learn and adopt. For example, it's assumed that most software developers understand SQL, whereas something like Gremlin (a graph query language) may be a niche skill. The higher the star rating, the easier the learning curve. The lower the star rating, the harder the learning curve.

Ease of data modeling

This characteristic refers to the ease with which data modelers can represent the domain in terms of a data model. A higher star rating means data modeling matches many use cases, and once modeled, is easy to change and adopt.

Scalability/throughput

This characteristic refers to the degree and ease with which a database can scale to handle increased throughput. Is it easy to scale the database? Can the database scale horizontally, vertically, or both? A higher star rating means it's easier to scale and get higher throughput.

Availability/partition tolerance

This characteristic refers to whether the database supports high availability configurations (such as *replica-sets* in MongoDB or *tunable consistency* in Apache Cassandra). Does it provide features to handle network partitions? The higher the

star rating, the better the database supports higher availability and/or better partition tolerance.

Consistency

This characteristic refers to whether the database supports an "always consistent" paradigm. Does the database support ACID transactions, or does it lean toward BASE transactions with an eventual consistency model? Does it provide features to have tunable consistency models for different types of writes? The higher the star rating, the more consistency the database supports.

Programming language support, product maturity, SQL support, and community

This characteristic refers to which (and how many) programming languages the database supports, how mature the database is, and the size of the database community. Can an organization easily hire people who know how to work with the database? Higher star ratings means there is better support, the product is mature, and it's easy to hire talent.

Read/write priority

This characteristic refers to whether the database prioritizes reads over writes, or writes over reads, or if it is balanced in its approach. This is not a binary choice—rather, it's more of a scale toward which direction the database optimizes.

Relational Databases

Relational databases (also known as an RDBMS) have been the database of choice for more than three decades. There is significant value in their usage and the stability they provide, particularly within most business-related applications. These databases are known for the ubiquitous Structured Query Language (SQL) and the ACID properties they provide. The SQL interface they provide makes them a preferred choice for implementing different `read` models on top of the same `write` model. The star ratings for relational databases appear in Figure 6-25.

Rating subject	RDBMS databases (Oracle, SQL Server, Postgres, etc.)
Ease of learning	★★★★
Ease of data modeling	★★★
Scalability/throughput	★★
Availability/partition tolerance	★
Consistency	★★★★★
Programming language support, product maturity, SQL support, community	★★★★
Read/write priority	Read — Write

Figure 6-25. Relational databases rated for various adoption characteristics

Ease-of-learning curve

Relational databases have been around for many years. They are commonly taught in schools, and mature documentation and tutorials exist. Therefore, they are much easier to learn than other database types.

Ease of data modeling

Relational databases allow for flexible data modeling. They allow the modeling of key-value, document, graph-like structures, and they allow for changes in read patterns with addition of new indexes. Some models are really difficult to achieve, such as graph structures with arbitrary depth. Relational databases organize data into tables and rows (similar to spreadsheets), something that is natural for most database modelers.

Scalability/throughput

Relational databases are generally vertically scaled using large machines. However, setup with replications and automated switchover are complex, requiring higher coordination and setup.

Availability/partition tolerance

Relational databases favor consistency over availability and partition tolerance, discussed in “Table Split Technique”.

Consistency

Relational databases have been dominant for years because of their support for ACID properties. The ACID features handle many concerns in concurrent systems and allow for developing applications without being concerned about lower-level details of concurrency and how the databases handle them.

Programming language support, product maturity, SQL support, and community

Since relational databases have been around for many years, well-known design, implementation, and operational patterns can be applied to them, thus making them easy to adopt, develop, and integrate within an architecture. Many of the relational databases lack support for reactive stream APIs and similar new concepts; newer architectural concepts take longer to implement in well-established relational databases. Numerous programming language interfaces work with relational databases, and the community of users is large (although splintered among all the vendors).

Read/write priority

In relational databases, the data model can be designed in such a way that either reads become more efficient or writes become more efficient. The same database can handle different types of workloads, allowing for balanced read-write priority. For example, not all use cases need ACID properties, especially in large data and traffic scenarios, or when really flexible schema is desired such as in survey administration. In these cases, other database types may be a better option.

MySQL, Oracle, Microsoft SQL Server, and PostgreSQL are the most popular relational databases and can be run as standalone installations or are available as *Database as a Service* on major cloud provider platforms.

AGGREGATE ORIENTATION

Aggregate orientation is the preference to operate on data that is related and has a complex data structure. Aggregate is a term originated in *Domain-Driven Design: Tackling Complexity in the Heart of Software* by Eric Evans. Think of `ticket` or `customer` with all its dependent tables in the Sysops Squad—they are aggregates. Like all practices in architecture, aggregate orientation includes benefits and shortcomings:

Benefits

- Enables easy distribution of data in clusters of servers, as the whole aggregate can be copied over to different servers.
- Improves read and write performance, as it reduces joins in the database.
- Reduces impedance mismatch between the application model and storage model.

Shortcomings

- It's difficult to arrive at proper aggregates, and changing aggregate boundaries is hard.
- Analyzing data across aggregates is difficult.

Key-Value Databases

Key-value databases are similar to a hash table data structure, something like tables in an RDBMS with an `ID` column as the key and a `blob` column as the value, which can consequently store any type of data. Key-value databases are part of a family known as NoSQL databases. In the book *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence* (Addison-Wesley), Pramod Sadalage (one of your authors) and Martin Fowler describe the rise of NoSQL databases and the motivations, usages, and trade-offs of using these types of databases, and is a good reference for further information on this database type.

Key-value databases are easiest to understand among the NoSQL databases. An application client can insert a key and a value, get a value for a known key, or delete a known key and its value. A key-value database does not know what's inside the value part, nor does it care what's inside, meaning that the database can query using the key and nothing else.

Unlike relational databases, key-value databases should be picked based on needs. There are persistent key-value databases like Amazon DynamoDB or Riak KV, nonpersistent databases like MemcacheDB, and other databases like Redis that can be configured to be persistent or not. Other relational database constructs like `joins`, `where`, and `order by` are not

supported, but rather the operations `get`, `put`, and `delete`. The ratings for key-value databases appear in Figure 6-26.

Rating subject	Key value databases (Redis, DynamoDB, Riak, etc.)
Ease of learning	★★★
Ease of data modeling	★
Scalability/throughput	★★★★
Availability/partition tolerance	★★★★
Consistency	★★
Programming language support, product maturity, SQL support, community	★★★
Read/write priority	Read ▲ Write

Figure 6-26. Key-value databases rated for various adoption characteristics

Ease-of-learning curve

Key-value databases are easy to understand. Since they use "Aggregate Orientation", it's important to design the aggregate properly because any change in the aggregate means rewriting all the data. Moving from relational databases to any of the NoSQL databases takes practice and unlearning familiar practices. For example, a developer cannot simply query "Get me all the keys."

Ease of data modeling

Since key-value databases are aggregate oriented, they can use memory structures like arrays, maps, or any other type of data, including big blob. The data can be queried only by key or ID, which means the client should have access to the key outside of the database. Good examples of a key include `session_id`, `user_id`, and `order_id`.

Scalability/throughput

Since key-value databases are indexed by key or ID, key lookups are very fast as there are no `joins` or `order by` operations. The value is fetched and returned to the client, which allows for easier scaling and higher throughput.

Availability/partition tolerance

Since there are many types of key-value databases and each has different properties, even the same database can be configured to act in different ways either for an installation or for each read. For example, in Riak users can use *quorum* properties such as `all`, `one`, `quorum`, and `default`. When we use *one* quorum, the query can return `success` when any one node responds. When the `all` quorum is used, all nodes have to respond for the query to return `success`. Each query can tune the partition tolerance and availability. Hence, assuming that all key-value stores are the same is a mistake.

Consistency

During each write operation, we can apply configurations that are similar to applying quorum during read operations; these configurations provide what is known as *tunable consistency*. Higher consistency can be achieved by trading off latency. For a write to be highly consistent, all nodes have to respond, which reduces partition tolerance. Using a *majority quorum* is considered a good trade-off.

Programming language support, product maturity, SQL support, and community

Key-value databases have good programming language support, and many open source databases have an active community to help learn and understand them. Since most databases have an *HTTP REST API*, they are much easier to interface with.

Read/write priority

Since key-value databases are aggregate oriented, access to data via a key or ID is geared toward *read* priority. Key-value databases can be used for session storage, and can be used to cache user properties and preferences as well.

SHARDING IN DATABASES

The concept of partitioning is well-known in relational databases: the table data is partitioned into sets based on a *schema* on the same database server. *Sharding* is similar to partitioning, but data resides on different servers or nodes. Nodes collaborate to figure out where data exists or where data should be stored based on a sharding key. The word *shard* means horizontal partition of data in a database.

Document Databases

Documents such as JSON or XML are the basis of document databases. Documents are human-readable, self-describing, hierarchical tree structures. Document databases are another type of NoSQL database, whose ratings appear in Figure 6-27. These databases un-

derstand the structure of the data and can index multiple attributes of the documents, allowing for better query flexibility.

Rating subject	Document databases (MongoDB, CouchDB, Marklogic, etc.)
Ease of learning	★★★
Ease of data modeling	★★★
Scalability/throughput	★★
Availability/partition tolerance	★★★
Consistency	★★
Programming language support, product maturity, SQL support, community	★★★
Read/write priority	Read ▲ Write

Figure 6-27. Document databases rated for various adoption characteristics

Ease-of-learning curve

Document databases are like key-value databases where the value is human readable. This makes learning the database much easier. Enterprises are used to dealing with documents, such as XML and JSON in different contexts, such as API payloads and JavaScript frontends.

Ease of data modeling

Just like key-value databases, data modeling involves modeling aggregates such as orders, tickets, and other domain objects. Document databases are forgiving when it comes to aggregate design, as the parts of the aggregate are queryable and can be indexed.

Scalability/throughput

Document databases are aggregate oriented and easy to scale. Complex indexing reduces scalability, and increased data size leads to a need for partitioning or sharding. Once sharding is introduced, it increases the complexity and also forces the selection of a sharding key.

Availability/partition tolerance

Like key-value databases, document databases can be configured for higher availability. The setup gets complicated when there are replicated clusters for sharded

collections. The cloud providers are trying to make these setups more usable.

Consistency

Some document databases have started supporting ACID transactions within a collection, but this may not work in some edge cases. Just like key-value databases, document databases provide the ability to tune the read and write operations using the quorum mechanism.

Programming language support, product maturity, SQL support, and community

Document databases are the most popular of the NoSQL databases, with an active user community, numerous online learning tutorials, and many programming language drivers that allow for easier adoption.

Read/write priority

Document databases are aggregate oriented and have secondary indexes to query, so these databases are favoring read priority.

SCHEMA-LESS DATABASES

One common theme in NoSQL databases is duplication of data and schema attribute names. No two entries have to be the same in terms of schema or attribute names. This introduces interesting change control dynamics and provides flexibility. The schema-less nature of the database is powerful, but it's important to understand that the data always has a schema even if it's implicit or defined elsewhere. The application needs to handle multiple versions of the schema returned by a database. The claim that NoSQL databases are entirely schema-less is misleading.

Column Family Databases

Column family databases, also known as *wide column databases* or *big table databases*, have rows with varying numbers of columns, where each column is a *name-value* pair. With columnar databases, the `name` is known as a *column-key*, the `value` is known as a *column-value*, and the primary key of a `row` is known as a *row key*. Column family databases are another type of NoSQL database that group related data that is accessed at the same time, and whose ratings appear in Figure 6-28.

Rating subject	Column family databases (Cassandra, Scylla, Druid, etc.)
Ease of learning	★★
Ease of data modeling	★
Scalability/throughput	★★★★
Availability/partition tolerance	★★★★
Consistency	★
Programming language support, product maturity, SQL support, community	★★
Read/write priority	Read — Write

Figure 6-28. Column family databases rated for various adoption characteristics

Ease-of-learning curve

Column family databases are difficult to understand. Since a collection of name-value pairs belong to a row, each row can have different name-value pairs. Some name-value pairs can have a map of columns and are known as *super columns*. Understanding how to use these takes practice and time.

Ease of data modeling

Data modeling with column family databases takes some getting used to. Data needs to be arranged in groups of name-value pairs that have a single row identifier, and designing this row key takes multiple iterations. Some column family databases like Apache Cassandra have introduced a SQL-like query language known as Cassandra Query Language (CQL) that makes data modeling accessible.

Scalability/throughput

All column family databases are highly scalable and suit use cases where high write or read throughput is needed. Column family databases scale horizontally for read and write operations.

Availability/partition tolerance

Column family databases naturally operate in clusters, and when some nodes of the cluster are down, it is transparent to the client. The default replication factor is three, which means at least three copies of data are made, improving availability and partition tolerance. Similar to key-value and document databases, column family databases can tune writes and reads based on quorum needs.

Consistency

Column family databases, like other NoSQL databases, follow the concept of tunable consistency. This means that, based on needs, each operation can decide how much consistency is desired. For example, in `high write` scenarios where some data loss can be tolerated, the write consistency level of `ANY` could be used, which means at least one node has accepted the write, while a consistency level of `ALL` means all nodes have to accept the write and respond success. Similar consistency levels can be applied to read operations. It's a trade-off—higher consistency levels reduce availability and partition tolerance.

Programming language support, product maturity, SQL support, and community

Column family databases like Cassandra and Scylla have active communities, and the development of SQL-like interfaces has made the adoption of these databases easier.

Read/write priority

Column family databases use the concepts of *SSTables*, *commit logs*, and *memtables*, and since the name-value pairs are populated when data is present, they can handle sparse data much better than relational databases. They are ideal for high write-volume scenarios.

All NoSQL databases are designed to understand aggregate orientation. Having aggregates improves read and write performance, and also allows for higher availability and partition tolerance when the databases are run as a cluster. The notion of CAP theorem is covered in "Table Split Technique" at more length.

Graph Databases

Unlike relational databases, where relations are implied based on references, graph databases use nodes to store entities and their properties. These nodes are connected with *edges*, also known as *relationships*, which are explicit objects. Nodes are organized by relationships and allow for analysis of the connected data by traversing along specific edges.



Figure 6-29. In graph databases, direction of the edge has significance when querying

The edges in graph databases have directional significance. In Figure 6-29, an edge of type `TICKET_CREATED` connecting a *ticket* node with ID `4235143` to a *customer* node with ID `Neal`. We can traverse from the ticket node via the *outgoing* edge `TICKET_CREATED` or the customer node via the *incoming* edge `TICKET_CREATED`. When the directions get mixed up, querying the graph becomes really difficult. The ratings for graph databases are illustrated in Figure 6-29.

Rating subject	Graph databases (Neo4J, Infinite Graph, Tigergraph, etc.)
Ease of learning	★
Ease of data modeling	★★
Scalability/throughput	★★★
Availability/partition tolerance	★★★
Consistency	★★★
Programming language support, product maturity, SQL support, community	★★
Read/write priority	Read ▲ Write

Figure 6-30. Graph databases rated for various adoption characteristics

Ease-of-learning curve

Graph databases have a steep learning curve. Understanding how to use the nodes, relations, relation type, and properties takes time.

Ease of data modeling

Understanding how to model the domains and convert them into nodes and relations is hard. In the beginning, the tendency is to add properties to relations. As modeling knowledge improves, increased usage of nodes and relations, and converting some relation properties to nodes with additional relation type takes place, which improves graph traversal.

Scalability/throughput

Replicated nodes improve read scaling, and throughput can be tuned for read loads. Since it's difficult to split or shard graphs, write throughput is constrained with the type of graph database picked. Traversing the relationships is very fast, as the indexing and storage is persisted and not calculated at query time.

Availability/partition tolerance

Some of the graph databases that have high partition tolerance and availability are distributed. Graph database clusters can use nodes that can be promoted as leaders when current leaders are unavailable.

Consistency

Many graph databases support ACID transactions. Some graph databases, such as Neo4j, support transactions, so that data is always consistent.

Programming language support, product maturity, SQL support, and community

Graph databases have lots of support in the community. Many algorithms, like Dijkstra's algorithm or *node similarity*, are implemented in the database, reducing the need to write them from scratch. The language framework known as Gremlin works across many different databases, helping in the ease of use. Neo4J supports a query language known as Cypher, allowing developers to easily query the database.

Read/write priority

In graph databases, data storage is optimized for relationship traversal as opposed to relational databases, where we have to query the relationships and derive them at query time. Graph databases are better for read-heavy scenarios.

Graph databases allow the same node to have various types of relationships. In the Sysops Squad example, a sample graph might look as follows: a `knowledge_base` was `created_by` user `sysops_user` and `knowledge_base` `used_by` `sysops_user`. Thus, the relationships `created_by` and `used_by` join the same nodes for different relationship types.

CHANGING RELATIONSHIP TYPES

Changing relationship types is an expensive operation, since each relationship type has to be re-created. When this happens, both nodes connected by the edge have to be visited, the new edge created, and the old edge removed. Hence, edge type or relationship types have to be thought about carefully.

NewSQL Databases

Matthew Aslett first used the term *NewSQL* to define new databases that aimed to provide the scalability of NoSQL databases while supporting the features of relational databases like ACID. NewSQL databases use different types of storage mechanisms, and all of them support SQL.

NewSQL databases, whose ratings appear in Figure 6-31, improve upon relational databases by providing automated data partitioning or sharding, allowing for horizontal scaling and improved availability, while at the same time allowing an easy transition for developers to use the known paradigm of SQL and ACID.

Rating subject	New SQL databases (VoltDB, NuoDB, ClustrixDB, etc.)
Ease of learning	★★★
Ease of data modeling	★★★
Scalability/throughput	★★★
Availability/partition tolerance	★★★
Consistency	★★
Programming language support, product maturity, SQL support, community	★★
Read/write priority	Read ▲ Write

Figure 6-31. New SQL databases rated for various adoption characteristics

Ease-of-learning curve

Since NewSQL databases are just like relational databases (with SQL interface, added features of horizontal scaling, ACID compliant), the learning curve is much easier. Some of them are available as only Database as a Service (DBaaS), which may make learning them more difficult.

Ease of data modeling

Since NewSQL databases are like relational databases, data modeling is familiar to many and easier to pick up. The extra wrinkle is sharding design, allowing sharded data placement in geographically different locations.

Scalability/throughput

NewSQL databases are designed to support horizontal scaling for distributed systems, allowing for multiple active nodes, unlike relational databases that have only one active leader, and the rest of the nodes are followers. The multiple active nodes allow NewSQL databases to be highly scalable and to have better throughput.

Availability/partition tolerance

Because of the multiple active nodes design, the benefits to availability can be really high with greater partition tolerance. CockroachDB is a popular NewSQL database that survives disk, machine, and data center failures.

Consistency

NewSQL databases support strongly consistent ACID transactions. The data is always consistent, and this allows for relational database users to easily transition to NewSQL databases.

Programming language support, product maturity, SQL support, and community

There are many open source NewSQL databases, so learning them is accessible. Some of the databases also support wire-compatible protocols with existing relational databases, which allows them to replace relational databases without any compatibility problems.

Read/write priority

NewSQL databases are used just like relational databases, with added benefits of indexing and distributing geographically either to improve read performance or write performance.

Cloud Native Databases

With increased cloud usage, cloud databases such as Snowflake, Amazon Redshift, Datomic, and Azure CosmosDB have gained in popularity. These databases reduce operational burden, provide cost transparency, and are an easy way to experiment since no upfront investments are needed. Ratings for cloud native databases appear in Figure 6-32.

Rating subject	Cloud databases (Snowflake, Amazon Redshift, etc.)
Ease of learning	★★
Ease of data modeling	★★
Scalability/throughput	★★★★
Availability/partition tolerance	★★★
Consistency	★★★
Programming language support, product maturity, SQLsupport, community	★★
Read/write priority	Read ▲ Write

Figure 6-32. Cloud native databases rated for various adoption characteristics

Ease-of-learning curve

Some cloud databases like AWS Redshift are like relational databases and therefore are easier to understand. Databases like Snowflake, which have a SQL interface but have different storage and compute mechanisms, require some practice. Datomic is totally different in terms of models and uses `immutable` atomic facts. Thus, the learning curve varies with each database offering.

Ease of data modeling

Datomic does not have the concept of tables or the need to define attributes in advance. It is necessary to define properties of individual attributes, and entities can have any attribute. Snowflake and Redshift are used more for data warehousing type workloads. Understanding the type of modeling provided by the database is critical in selecting the database to use.

Scalability/throughput

Since all these databases are cloud only, scaling them is relatively simple since resources can be allocated automatically for a price. In these decisions, the trade-off typically relates to price.

Availability/partition tolerance

Databases in this category (such as Datomic) are highly available when deployed using *Production Topology*. They have no single point of failure and are supported by extensive caching. Snowflake, for example, replicates its databases across regions and accounts. Other databases in this category support higher availability with vari-

ous options to configure. For example, Redshift runs in a single availability zone and would need to be run in multiple clusters to support higher availability.

Consistency

Datomic supports ACID transactions using storage engines to store blocks in block storage. Other databases, like Snowflake and Redshift, support ACID transactions.

Programming language support, product maturity, SQL support, and community

Many of these databases are new, and finding experienced help can be difficult. Experimenting with these databases requires a cloud account, which can create another barrier. While cloud native databases reduce operational workload on the operational DBAs, they do have a higher learning curve for developers. Datomic uses Clojure in all its examples, and stored procedures are written with Clojure, so not knowing Clojure maybe a barrier to usage.

Read/write priority

These databases can be used for both read-heavy or write-heavy loads. Snowflake and Redshift are geared more toward data warehouse type workloads, lending them toward read priority, while Datomic can support both type of loads with different indexes such as EAVT (Entity, Attribute, Value, then Transaction) first.

Time-Series Databases

Given the trends, we see increased usage of IoT, microservices, self-driving cars, and observability, all of which have driven a phenomenal increase in time-series analytics. This trend has given rise to databases optimized for storing sequences of data points collected during a time window, enabling users to track changes over any duration of time. The ratings for this database type appear in Figure 6-33.

Rating subject	Time series databases (InfluxDB, TimescaleDB, etc.)
Ease of learning	★
Ease of data modeling	★★
Scalability/throughput	★★★★
Availability/partition tolerance	★★
Consistency	★★★
Programming language support, product maturity, SQL support, community	★★
Read/write priority	Read ▲ Write

Figure 6-33. Time-series databases rated for various adoption characteristics

Ease-of-learning curve

Understanding time-series data is often easy—every data point is attached to a time-stamp, and data is almost always inserted and never updated or deleted. Understanding append-only operations takes some unlearning from other database usage, where errors in the data can be corrected with an update. InfluxDB, Kx, and TimeScale are some of the popular time-series databases.

Ease of data modeling

The underlying concept with time-series databases is to analyze changes in data over time. For example, with the Sysops Squad example, changes done to a ticket object can be stored in a time-series database, where the `timestamp` of change and `ticket_id` are tagged. It's considered bad practice to add more than one piece of information in one tag. For example, `ticket_status=Open, ticket_id=374737` is better than `ticket_info=Open.374737`.

Scalability/throughput

Timescale is based on PostgreSQL and allows for standard scaling and throughput improvement patterns. Running InfluxDB in cluster mode by using *meta* nodes that manage metadata and *data* nodes that store actual data provides scaling and throughput improvements.

Availability/partition tolerance

Some databases like InfluxDB have better availability and partition tolerance options with configurations for meta and data nodes, along with *replication* factors.

Consistency

Time-series databases that use relational databases as their storage engine get ACID properties for consistency, while other databases can tune consistency using `consistency-level` of `any`, `one`, or `quorum`. Higher consistency-level configuration generally results in higher *consistency* and lower *availability*, so it's a trade-off that needs to be considered.

Programming language support, product maturity, SQL support, and community

Time-series databases have become popular lately, and there are many resources to learn from. Some of these databases, such as InfluxDB, provide a SQL-like query language known as InfluxQL.

Read/write priority

Time-series databases are append only and tend to be better suited for read-heavy workloads.

When using time-series databases, the database automatically attaches a timestamp to every datum creation, and the data contains tags or attributes of information. The data is queried based on some fact between specific time windows. Therefore, time-series databases are not general-purpose databases.

In summary of all the database types discussed in this section, Table 6-6 shows some popular database products for the database type.

Table 6-6. Summary of database types and products in the database type

Database type	Products
Relational	PostgreSQL, Oracle, Microsoft SQL
Key-value	Riak KV, Amazon DynamoDB, Redis
Document	MongoDB, Couchbase, AWS DocumentDB
Column family	Cassandra, Scylla, Amazon SimpleDB
Graph	Neo4j, Infinite Graph, Tiger Graph
NewSQL	VoltDB, ClustrixDB, SimpleStore (aka MemSQL)
Cloud native	Snowflake, Datomic, Redshift
Time-series	InfluxDB, kdb+, Amazon Timestream

Sysops Squad Saga: Polyglot Databases

Thursday, December 16, 16:05



Now that the team had formed data domains from the monolithic Sysops Squad database, Devon noticed that the Survey data domain would be a great candidate for migrating from a traditional relational database to a document database using JSON. However, Dana, the head of data architecture, didn't agree and wanted to keep the tables as relational.

"I simply don't agree," said Dana. "The survey tables have always worked in the past as relational tables, so I see no reason to change things around. "

"Actually," said Skyler, "if you had originally talked with us about this when the system was first being developed, you would understand that from a user interface perspective, it's really hard to deal with relational data for something like a customer survey. So I disagree. It may work out good for you, but from a user interface development standpoint, dealing with relational data for the survey stuff has been a major pain point."

"See, so there you are," said Devon. "This is why we need to change it to a document database."

“You seem to forget that as the data architect for this company, I am the one who has ultimate responsibility for all these different databases. You can’t just start adding different database types to the system,” said Dana.

“But it would be a much better solution,” said Devon.

“Sorry, but I’m not going to cause a disruptor on the database teams just so Skyler can have an easier job maintaining the user interface. Things don’t work that way.”

“Wait,” said Skyler, “didn’t we all agree that part of the problem of the current monolithic Sysops Squad application was that the development teams didn’t work close enough with the database teams?”

“Yes,” said Dana.

“Well then,” said Skyler, “let’s do that. Let’s work together to figure this out.”

“OK,” said Dana, “but what I’m going to need from you and Devon is a good solid justification for introducing another type of database into the mix.”

“You got it,” said Devon. “We’ll start working on that right away.”

Devon and Skyler knew that a document database would be a much better solution for the customer survey data, but they weren’t sure how to build the right justifications for Dana to agree to migrate the data. Skyler suggested that they meet with Addison to get some help, since both agreed that this was somewhat an architectural concern. Addison agreed to help, and set up a meeting with Parker (the Sysops Squad product owner) to validate whether there was any business justification to migrating the customer survey tables to a document database.

“Thanks for meeting with us, Parker,” said Addison. “As I mentioned to you before, we are thinking of changing the way the customer survey data is stored, and have a few questions for you.”

“Well,” said Parker, “that was one of the reasons why I agreed to this meeting. You see, the customer survey part of the system has been a major pain point for the marketing department, as well as for me.”

“Huh?” asked Skyler. “What do you mean?”

“How long does it take you to apply even the smallest of change requests to the customer surveys?” asked Parker.

"Well," said Devon, "it's not too bad from the database side. I mean, it's a matter of adding a new column for a new question or changing the answer type."

"Hold on," said Skyler. "Sorry, but for me it's a major change, even when you add an additional question. You have no idea how hard it is to query all of that relational data and render a customer survey in the user interface. So, my answer is, a *very long time*."

"Listen," said Parker. "We on the business side of things get very frustrated ourselves when even the simplest of changes take you literally days to do. It's simply not acceptable."

"I think I can help here," said Addison. "So Parker, what you're saying is that the customer survey changes frequently, and it is taking too long to make the changes?"

"Correct," said Parker. "The marketing department not only wants better flexibility in the customer surveys, but better response from the IT department as well. Many times they don't place change requests because they know it will just end in frustration and additional cost they didn't plan for."

"What if I were to tell you that the lack of flexibility and responsiveness to change requests has everything to do with the technology used to store customer surveys, and that by changing the way we store data, we could significantly improve flexibility as well as response time for change requests?" asked Addison.

"Then I would be the happiest person on Earth, as would the marketing department," said Parker.

"Devon and Skyler, I think we have our business justification," said Addison.

With the business justification established, Devon, Skyler, and Addison convinced Dana to use a document database. Now the team had to figure out the optimal structure for the customer survey data. The existing relational database tables are illustrated in Figure 6-34. Each customer survey consisted of two primary tables—a Survey table and a Question table, with a one-to-many relationship between the two tables.

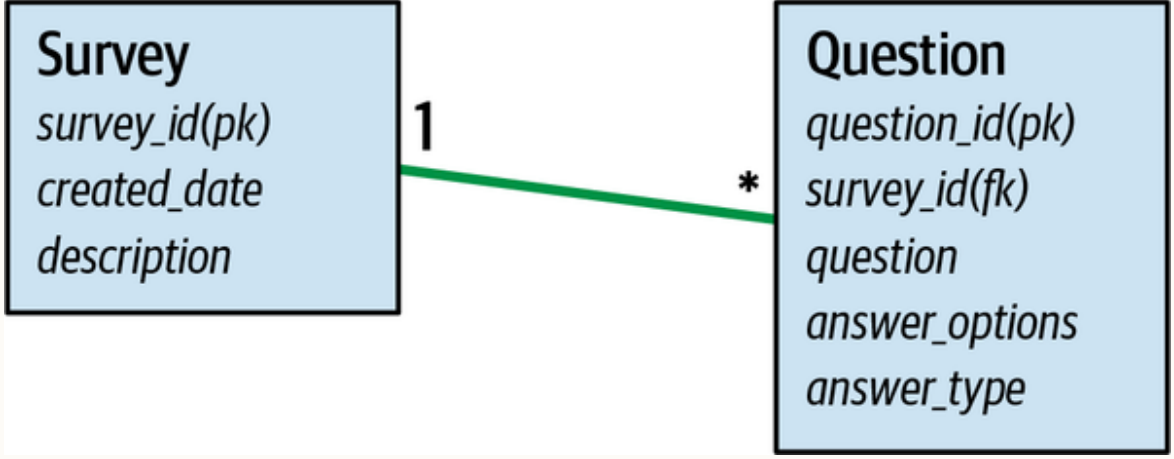


Figure 6-34. Tables and relationships in the sysops survey data domain

An example of the data contained in each table is shown Figure 6-35, where the Question table contains the question, the answer options, and the data type for the answer.

Survey

survey_id	created_date	description
19998	May 20 2022	Expert performance survey.
19999	May 20 2022	Service satisfaction survey.

Question

question_id	survey_id	question	answer_options	answer_type
50000	19999	Did the..	{Yes,No}	Boolean
50001	19999	Rate..	{1,2,3,4,5}	Option

Figure 6-35. Relational data in tables for survey and question in the survey data domain

"So, essentially we have two options for modeling the survey questions in a document database," said Devon. "A single aggregate document or one that is split."

"How to we know which one to use?" asked Skyler, happy that the development teams were now finally working with the database teams to arrive at a unified solution.

"I know," said Addison, "let's model both so we can visually see the trade-offs with each approach."

Devon showed the team that with the single aggregate option, as shown in Figure 6-36, with the corresponding source code listing in Example 6-3, both the survey data and all related question data were stored as one document. Therefore, the entire customer survey could be retrieved from the database by using a single `get` operation, making it easy for Skyler and others on the development team to work with the data.

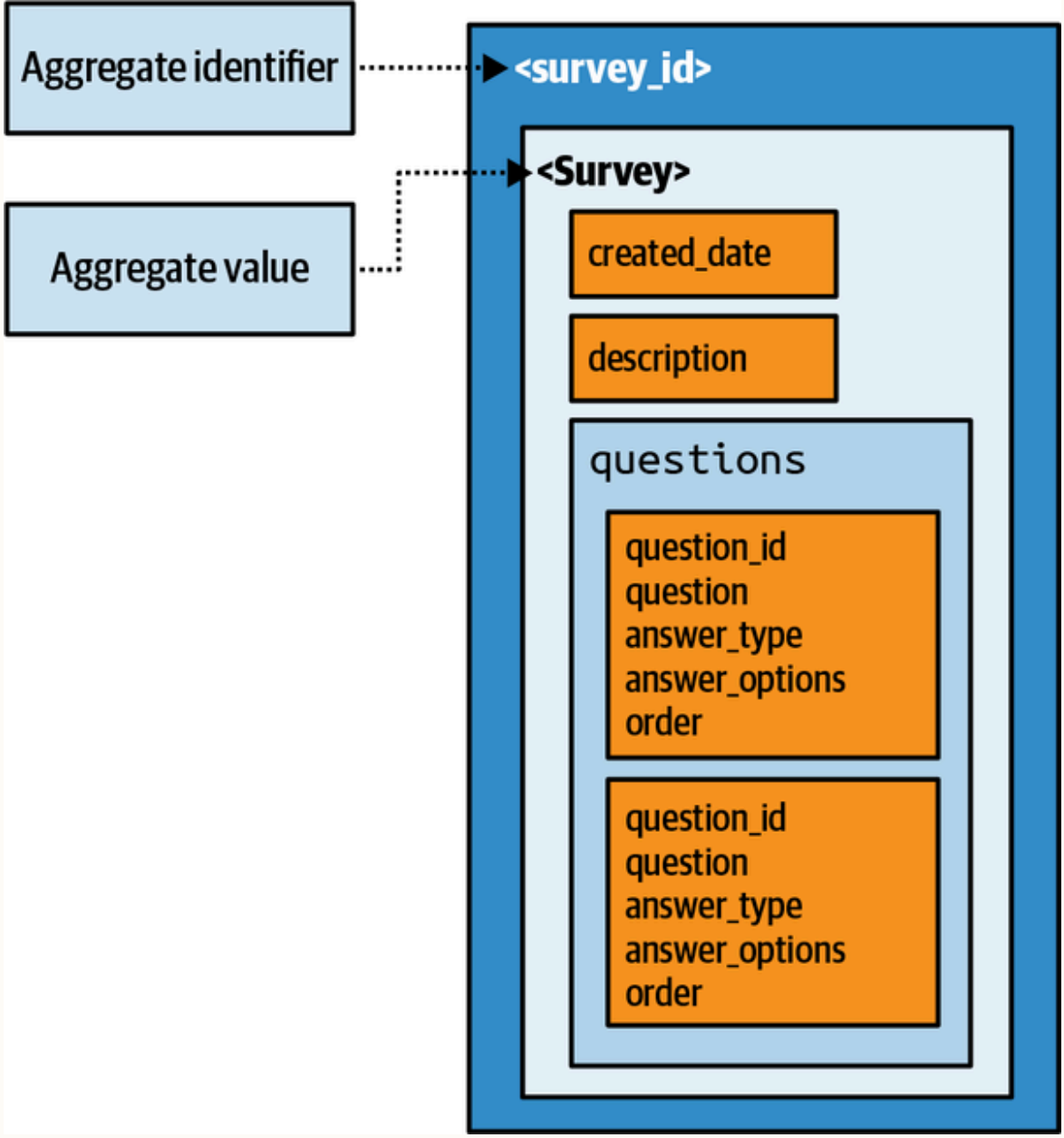


Figure 6-36. Survey model with single aggregate

Example 6-3. JSON document for single aggregate design with children embedded

```
# Survey aggregate with embedded questions
{
   "survey_id": "19999",
   "created_date": "Dec 28 2021",
   "description": "Survey to gauge customer...",
   "questions": [
      {
         "question_id": "50001",
         "question": "Rate the expert",
         "answer_type": "Option",
         "answer_options": "1,2,3,4,5",
         "order": "2"
      },
      {
         "question_id": "50000",
         "question": "Did the expert fix the problem?",
```

```
        "answer_type": "Boolean",
        "answer_options": "Yes,No",
        "order": "1"
      }
    ]
  }
```

"I really like that approach," said Skyler. "Essentially, I wouldn't have to worry so much about aggregating things myself in the user interface, meaning I could simply render the document I retrieve on the web page."

"Yeah," said Devon, "but it would require additional work on the database side as questions would be replicated in each survey document. You know, the whole reuse argument. Here, let me show you the other approach."

Skyler explained that another way to think about aggregates was to split the *survey* and *question* model so that the questions could be operated on in an independent fashion, as shown in Figure 6-37, with the corresponding source code listing in Example 6-4. This would allow the same question to be used in multiple surveys, but would be harder to render and retrieve than the single aggregate.

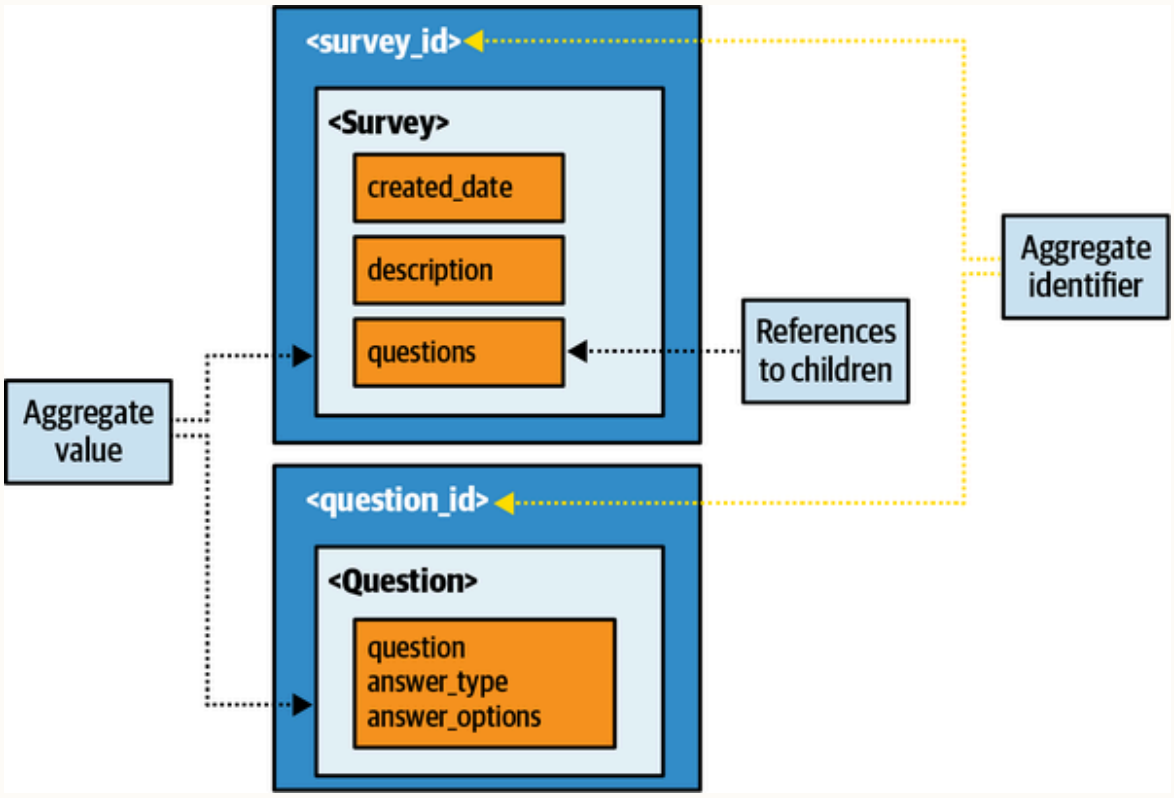


Figure 6-37. Survey model with multiple aggregates with references

Example 6-4. JSON document with aggregates split and parent document showing references to children

```
# Survey aggregate with references to Questions
{
   "survey_id": "19999",
   "created_date": "Dec 28",
   "description": "Survey to gauge customer...",
   "questions": [
      {"question_id": "50001", "order": "2"},
      {"question_id": "50000", "order": "1"}
   ]
}
# Question aggregate
{
   "question_id": "50001",
   "question": "Rate the expert",
   "answer_type": "Option",
   "answer_options": "1,2,3,4,5"
}
{
   "question_id": "50000",
   "question": "Did the expert fix the problem?",
   "answer_type": "Boolean",
   "answer_options": "Yes,No"
}
```

Because most of the complexity and change issues were in the user interface, Skyler liked the single aggregate model better. Devon liked the multiple aggregate to avoid duplication of question data in each survey. However, Addison pointed out that there were only five survey types (one for each product category), and that most of the changes involved adding or removing questions. The team discussed the trade-offs, and all agreed that they were willing to trade off some duplication of question data for the ease of changes and rendering on the user interface side. Because of the difficulty of this decision and the structural nature of changing the data, Addison created an ADR to record the justifications of this decision:

ADR: Use of Document Database for Customer Survey

Context
Customers receive a survey after the work has been completed by the customer, which is rendered on a web page for the customer to fill out and submit. The customer receives one of five survey types based on the type of electronic product fixed or installed. The survey is currently stored in a relational database, but the team wants to migrate the survey to a document database using JSON.

Decision
We will use a document database for the customer survey.

The Marketing Department requires more flexibility and timeliness for changes to the customer surveys. Moving to a document database would not only provide better flexibility, but also better timeliness for changes needed to the customer surveys.

Using a document database would simplify the customer survey user interface and better facilitate changes to the surveys.

Consequences
Since we will be using a single aggregate, multiple documents would need to be changed when a common survey question is updated, added, or removed.

Survey functionality will need to be shut down during the data migration from the relational database to the document database.

Previous chapter
5. Component-Based Decomposition Patterns

Next chapter
7. Service Granularity